



Variable Selection (=Feature Selection)

빅데이터분석
컴퓨터AI공학부
천세진

Variable Selection(변수 선택)

- 강의 목표: 다른 변수선택 기법을 이해하는 것이 목표
- 변수선택 기법: 데이터셋 내 제일 좋은 특징(features)를 선택하는 방법
- 데이터 과학에서 핵심 과정 중의 하나임





Feature vector

Feature vector: Collection of numerical features

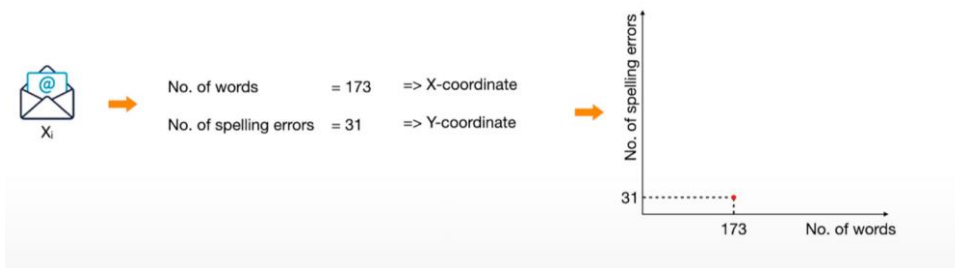
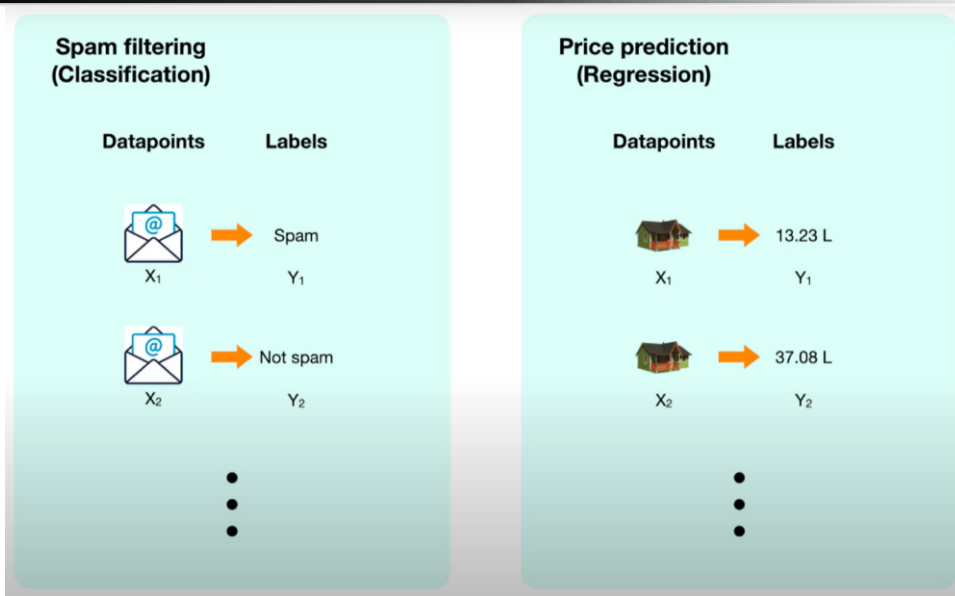
dwa

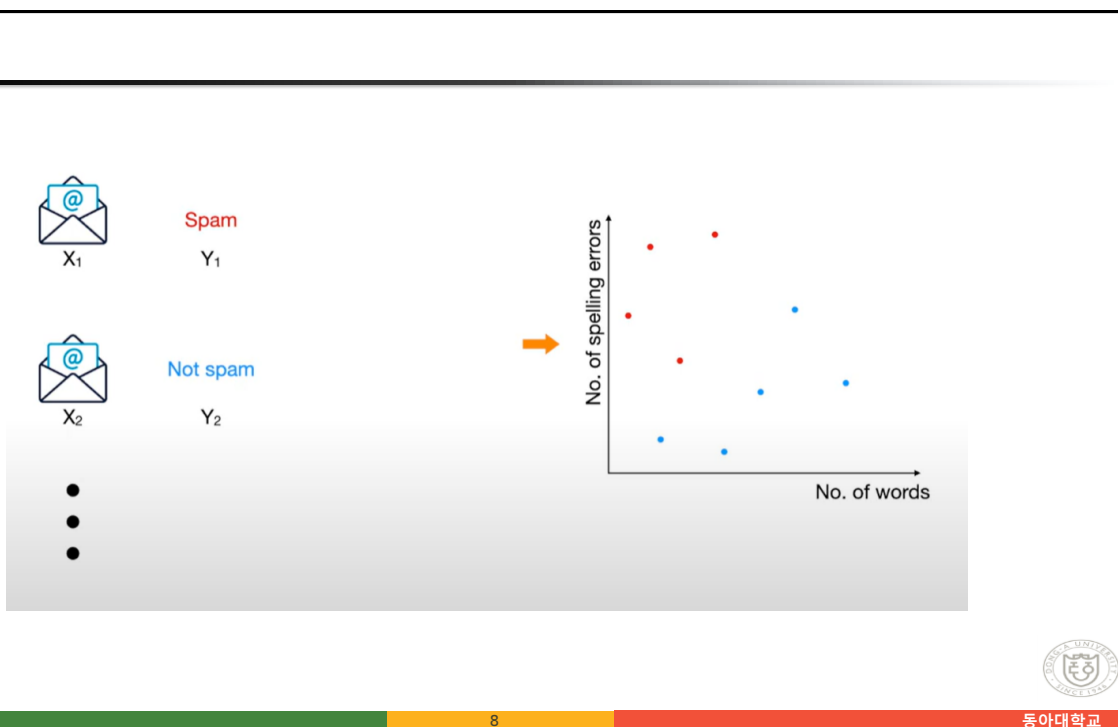
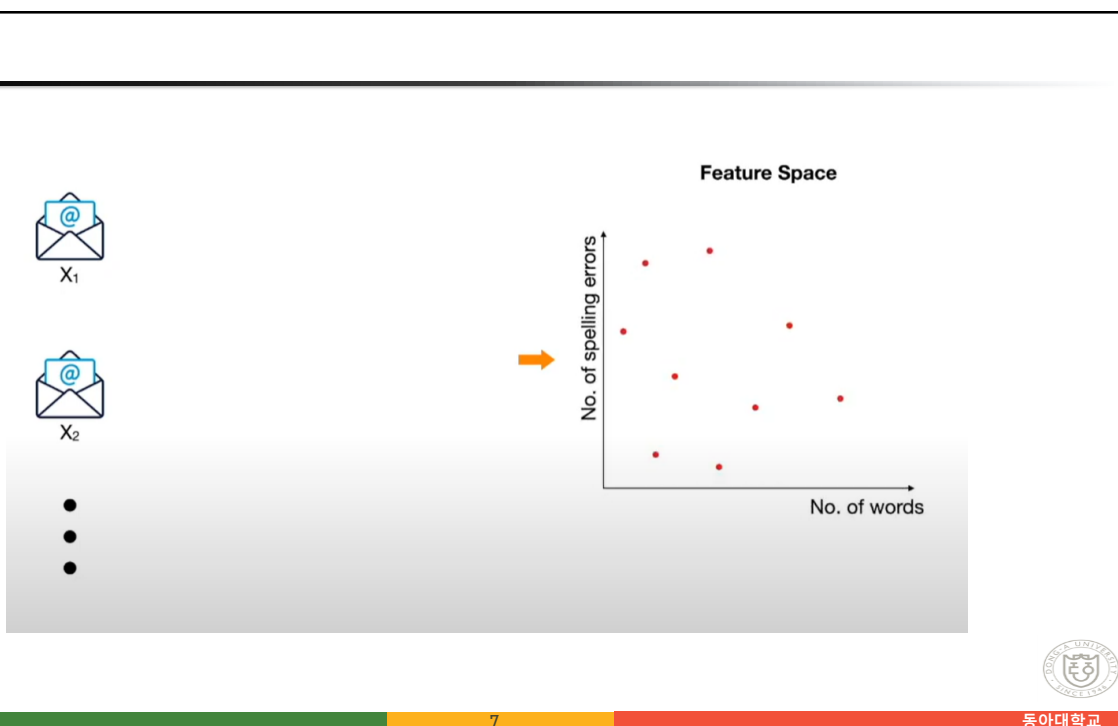
Feature vector: Collection of numerical features

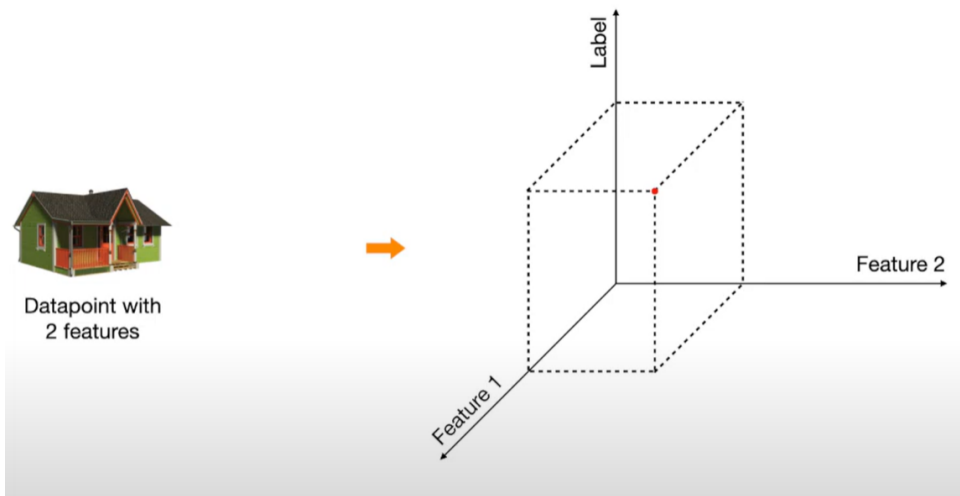
	No. of words No. of "the" Does the word "sell" appear? No. of spelling errors ...	= 173 = 61 = No = 13 ...	173 61 0 13	→ [173, 61, 0, 13]
	No. of blue pixels Average brightness level RGB value of pixel at position (37, 71) Is there a dog in the image? ...	= 31,234 = 71 = (23, 97, 11) = No ...	31,234 71 (23, 97, 11) 0	→ [31234, 71, 23, 97, 11, 0]
	No. of rooms Area in sq ft. Distance to the nearest market Gender of current owner ...	= 3 = 1000 = 1 km = Female ...	3 1000 1 1	→ [3, 1000, 1, 1]
	No. of black pieces Position of white king Distance between black and white queens Which positions can the white queen attack? ...	= 15 = g1 = 4 = {b6, d2, e7} ...	15 63 4 771	→ [15, 63, 4, 771]



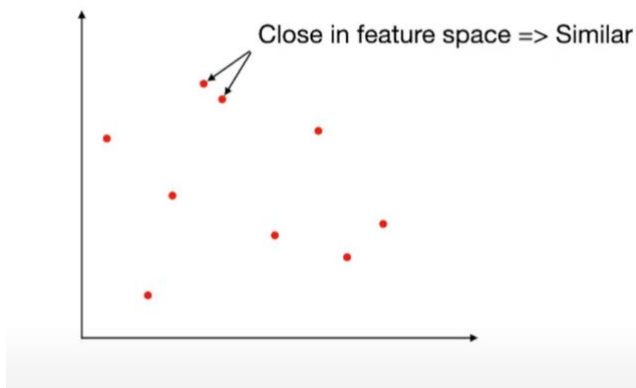
Feature space



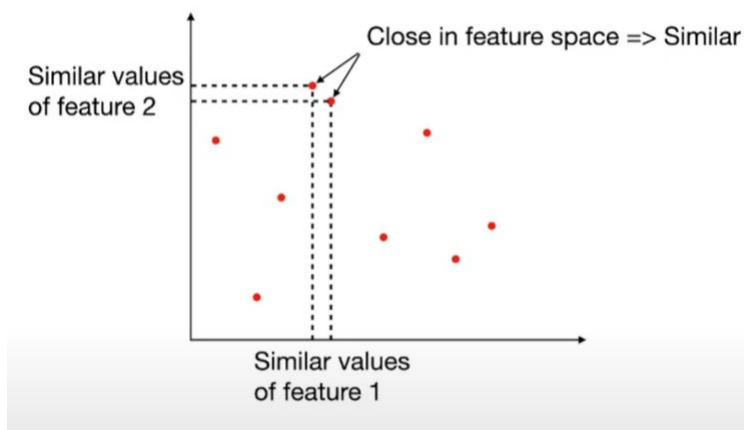




Property of Feature space



Property of Feature space

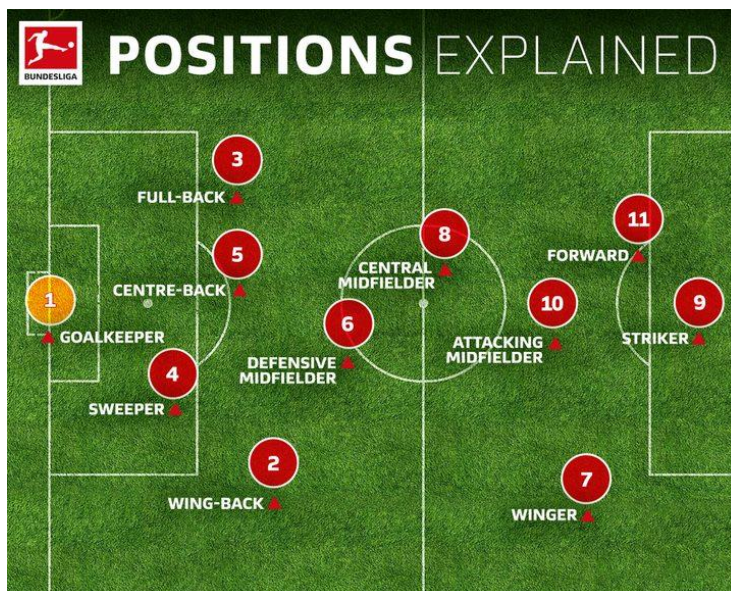


Feature selection

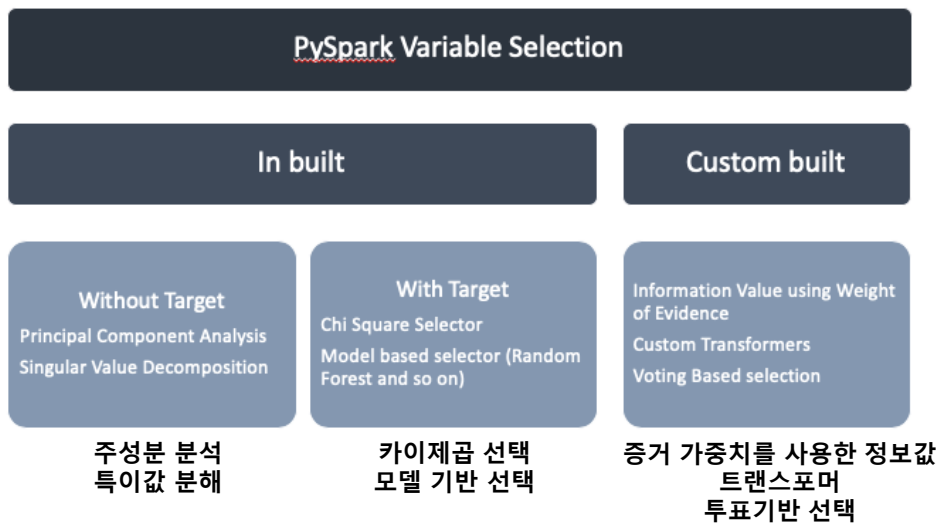
13

You are the Coach for Soccer team

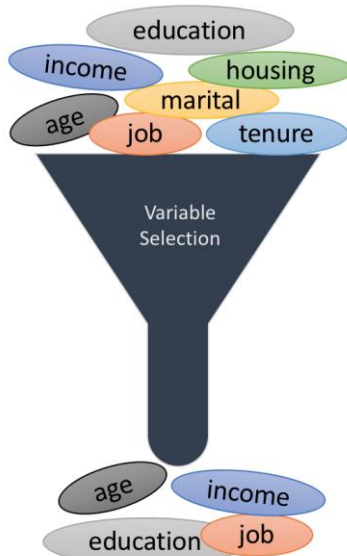
- The best player in each position
(Best features)
 - Don't want many players play the same position
(Multicollinearity)
- 다중공선성 문제



PySpark 기반 변수선택 기법



변수선택 기법의 직관적인 표현



Exploratory Data Analysis(EDA)

- 탐색적 데이터 분석: 모델링 전에 필수적인 활동
 - 데이터 내 존재하는 특징(Characteristics)과 패턴(patterns)을 식별하기 위해 데이터를 분석하는 과정
- Cardinality와 Missing Values
 - 데이터가 주어졌을 때, 먼저해야 체크해야 하는 부분



Cardinality

- 변수의 고유한 값에 대한 개수
 - 집 값을 예측하는 프로그램을 모델링
 - Home type은 유용한 정보인가?

Zip	Home type	HOA fees	Sq. Ft	Price
19102	Town Home	\$100	2400	\$350K
19103	Condo	\$300	1200	\$200K
19145	Single Family Unit	NaN	2500	\$400K
19150	Single Family Unit	NaN	1800	NaN
19040	Town Home	\$250	2275	\$325K

Home price dataset



Missing Values(결측값)

■ 정보의 사라진 부분

- Missing at random (MAR): 무작위 결측
 - 관측된 데이터와 결측된 데이터 간 관계, 예) Home type and HOA fees
- Missing completely at random(MCAR): 완전 무작위 결측
 - 관측된 데이터와 결측된 데이터 간 관계가 없음, 예) Price
- Missing not at random(MNAR): 비무작위 결측
 - 관측된 데이터와 결측된 데이터 간 관계가 있음

Zip	Home type	HOA fees	Sq. Ft	Price
19102	Town Home	\$100	2400	\$350K
19103	Condo	\$300	1200	\$200K
19145	Single Family Unit	NaN	2500	\$400K
19150	Single Family Unit	NaN	1800	NaN
19040	Town Home	\$250	2275	\$325K



Missing Values

■ 정보의 사라진 부분

X
|
Y

Z
|
R

(a) MCAR

X
|
Y

Z
|
R

|
|
|

(b) MAR

X
|
Y

Z
|
R

|
|
|

(c) MNAR



MAR의 경우에 대한 처리 방법

- 결측된 값을 채우는 방법을 사용
 - 데이터 대치법(Imputation) 혹은 데이터 보간법(Interpolation)
- 다음과 같은 형태를 고려해야함
 - Mean, median, or mode imputation
 - Model-based imputation
 - Multiple imputation
 - 비즈니스 로직 사용
 - Drop features with significant missing data (variable selection)
 - Drop rows with missing values(추천 안됨)



MNAR

- 데이터는 수집되지 않았음
 - 예로, built year
 - 데이터 수집과정에서는 존재하지 않는 값
 - 새로운 특징(features)를 찾는 과정

Zip	Home type	HOA fees	Sq. Ft	Price
19102	Town Home	\$100	2400	\$350K
19103	Condo	\$300	1200	\$200K
19145	Single Family Unit	<u>NaN</u>	2500	\$400K
19150	Single Family Unit	<u>NaN</u>	1800	<u>NaN</u>
19040	Town Home	\$250	2275	\$325K



Dataset 다운로드

■ <https://archive.ics.uci.edu/ml/datasets/Bank+Marketing>



UCI
Machine Learning Repository
Center for Machine Learning and Intelligent Systems

Check out the beta version of the new UCI Machine Learning Repository we are currently testing! Contact us if you have any issues, questions, or concerns. [Click here to try out the new site.](#)

Bank Marketing Data Set
Download [Data Folder](#) [Data Set Description](#)

Abstract: The data is related with direct marketing campaigns (phone calls) of a Portuguese banking institution. The classification goal is to predict if the client will subscribe a term deposit (variable y).

Data Set Characteristics:	Multivariate	Number of Instances:	45211	Area:	Business
Attribute Characteristics:	Real	Number of Attributes:	17	Date Donated:	2012-02-14
Associated Tasks:	Classification	Missing Values?	N/A	Number of Web Hits:	1615692

Source:
[Moro et al., 2014] S. Moro, P. Cortez and P. Rita. A Data-Driven Approach to Predict the Success of Bank Telemarketing. Decision Support Systems, Elsevier, 62:22-31, June 2014

Data Set Information:
The data is related with direct marketing campaigns of a Portuguese banking institution. The marketing campaigns were based on phone calls. Often, more than one contact to the same client was required, in order to achieve the client's engagement.

There are four datasets:
1) bank-additional-full.csv with all examples (41108) and 20 inputs, ordered by date (from May 2008 to November 2010), very close to the data analyzed in [Moro et al., 2014]
2) bank-additional.csv with 10% of the examples (4119), randomly selected from 1), and 20 inputs
3) bank-full.csv with all examples and 17 inputs, ordered by date (older version of this dataset with less inputs)
4) bank.csv with 10% of the examples and 17 inputs, randomly selected from 3) (older version of this dataset with less inputs)

The smallest datasets are provided to test more computationally demanding machine learning algorithms (e.g., SVM).

The classification goal is to predict if the client will subscribe (yes/no) a term deposit (variable y).

Bank Marketing Dataset

- 약 9만 9천 건의 데이터셋
- 고객정보가 주어졌을때, 우리 마케팅 프로그램에 가입 가능성?
- x: 고객정보, y: 마케팅 프로그램 가입여부

Input variables:
bank client data:
1 - age (numeric)
2 - job : type of job (categorical: 'admin', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self-employed', 'services', 'student', 'technician', 'unemployed', 'unknown')
3 - marital : marital status (categorical: 'divorced', 'married', 'single', 'unknown'; note: 'divorced' means divorced or widowed)
4 - education (categorical: 'basic.4y', 'basic.6y', 'basic.9y', 'high.school', 'illiterate', 'professional.course', 'university.degree', 'unknown')
5 - default: has credit in default? (categorical: 'no', 'yes', 'unknown')
6 - housing: has housing loan? (categorical: 'no', 'yes', 'unknown')
7 - loan: has personal loan? (categorical: 'no', 'yes', 'unknown')
related with the last contact of the current campaign:
8 - contact: contact communication type (categorical: 'cellular', 'telephone')
9 - month: last contact month of year (categorical: 'jan', 'feb', 'mar', ..., 'nov', 'dec')
10 - day_of_week: last contact day of the week (categorical: 'mon', 'tue', 'wed', 'thu', 'fri')
11 - duration: last contact duration, in seconds (numeric). Important note: this attribute highly affects the output target (e.g., if duration=0 then y='no'). Yet, the duration is not known before a call is performed. Also, after the end of the call y is obviously known. Thus, this input should only be included for benchmark purposes and should be discarded if the intention is to have a realistic predictive model.
other attributes:
12 - campaign: number of contacts performed during this campaign and for this client (numeric, includes last contact)
13 - pdays: number of days that passed by after the client was last contacted from a previous campaign (numeric, 999 means client was not previously contacted)
14 - previous: number of contacts performed before this campaign and for this client (numeric)
15 - poutcome: outcome of the previous marketing campaign (categorical: 'failure', 'nonexistent', 'success')
social and economic context attributes
16 - emp.var.rate: employment variation rate - quarterly indicator (numeric)
17 - cons.price.idx: consumer price index - monthly indicator (numeric)
18 - cons.conf.idx: consumer confidence index - monthly indicator (numeric)
19 - euribor3m: euribor 3 month rate - daily indicator (numeric)
20 - nr.employed: number of employees - quarterly indicator (numeric)

Output variable (desired target):
21 - y - has the client subscribed a term deposit? (binary: 'yes', 'no')

Bank Marketing Dataset

	Attributes	Description
Bank client data		
1	Age (numeric)	
2	Job (categorical)	Job descriptions
3	Marital (categorical)	Married, divorced, etc.
4	Education (categorical)	Unknown, secondary, primary, tertiary
5	Default	Yes or no (has credit card?)
6	Balance (numeric)	Average yearly balance
7	Housing loan	Yes/no
8	Personal loan	Yes/no
Related with the last contact of the current campaign		
9	Contact type (categorical)	Unknown, telephone, cellular
10	Day (numeric)	Last contact day of the month
11	Month (categorical)	Last contact month of year
12	Duration (numeric)	Last contact duration (second)
Other attributes		
13	Campaign (numeric)	Number of contacts performed during this campaign and for this client (numeric, includes last contact)
14	Pdays (numeric)	Number of days that passed by after the client was last contacted from a previous campaign
15	Previous (numeric)	Number of contacts performed before this campaign
16	Poutcome (categorical)	Outcome of the previous marketing campaign
Output variable (desired target)		
17	Y (binary)	Has the client subscribed a term deposit? (yes/ no)



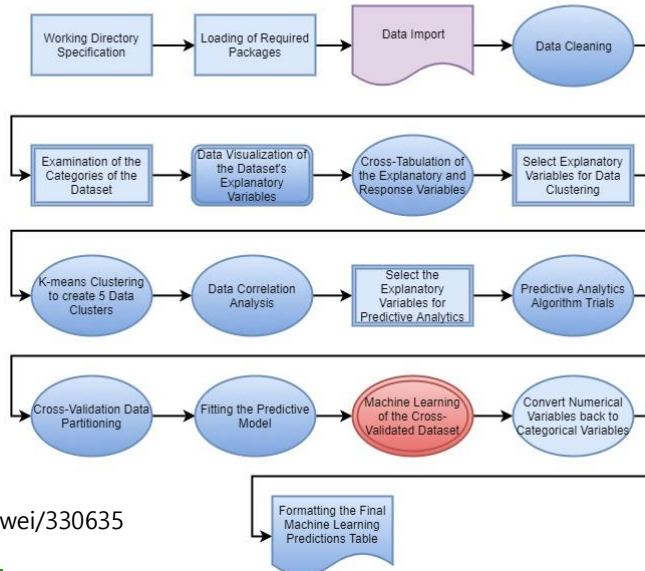
Bank Marketing Dataset

```
df = pd.read_csv('bank-full.csv', sep=';')
df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no



Bank Marketing Data Classification Flowchart



<https://rpubs.com/johnakwei/330635>



변수선택 기법

주어진 고객 데이터에 대한 검증

- Cardinality Check
- Missing Value Check
- String values
- Data Scaling



Quick setup for pySpark

- ✓ [1]

```
1 !pip install pyspark
2 !apt install openjdk-8-jdk-headless -qq
3 import os
4 os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
```
- ✓ [2]

```
1 # pyspark 모듈 및 sql 라이브러리 설치
2 import pyspark
3 from pyspark.sql import *
4 from pyspark.sql.functions import *
5 from pyspark import SparkContext, SparkConf
```
- ✓ [3]

```
1 # create the session
2 conf = SparkConf().set("spark.ui.port", "4050")
3
4 # create the context
5 sc = pyspark.SparkContext(conf=conf)
6 spark = SparkSession.builder.getOrCreate()
```
- ✓ [4]

```
1 spark
```



동아대학교

Load Dataset

<https://archive.ics.uci.edu/ml/datasets/Bank+Marketing>

- [5]

```
1 !wget -q https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank.zip
```
- ✓ [8]

```
1 !mkdir data
2 !unzip bank.zip -d data/
```

데이터 읽기

- ✓ [9]

```
1 filename = "data/bank-full.csv"
2 target_variable_name = "y"
```
- ▶

```
1 df = spark.read.csv(filename, header=True, inferSchema=True, sep=';')
2 df.show()
3 # df.schema 확인
```



컴퓨터공학부

30

동아대학교

InferSchema

■ df.printSchema()

● df.dtypes

Without inferSchema

```
[('age', 'string'),
 ('job', 'string'),
 ('marital', 'string'),
 ('education', 'string'),
 ('default', 'string'),
 ('balance', 'string'),
 ('housing', 'string'),
 ('loan', 'string'),
 ('contact', 'string'),
 ('day', 'string'),
 ('month', 'string'),
 ('duration', 'string'),
 ('campaign', 'string'),
 ('pdays', 'string'),
 ('previous', 'string'),
 ('poutcome', 'string'),
 ('y', 'string')]
```

With inferSchema

```
[('age', 'int'),
 ('job', 'string'),
 ('marital', 'string'),
 ('education', 'string'),
 ('default', 'string'),
 ('balance', 'int'),
 ('housing', 'string'),
 ('loan', 'string'),
 ('contact', 'string'),
 ('day', 'int'),
 ('month', 'string'),
 ('duration', 'int'),
 ('campaign', 'int'),
 ('pdays', 'int'),
 ('previous', 'int'),
 ('poutcome', 'string'),
 ('y', 'string')]
```



Descriptive Statistics

■ df.describe().toPandas() → 히스토그램을 사용하여 그래프를 그릴 수 있다.

1 df.describe().toPandas()

	summary	age	job	marital	education	default	balance	housing	loan	contact
0	count	45211	45211	45211	45211	45211	45211	45211	45211	45211
1	mean	40.93621021432837	None	None	None	None	1362.2720576850766	None	None	None
2	stddev	10.618762040975408	None	None	None	None	3044.7658291685243	None	None	None
3	min	18	admin.	divorced	primary	no	-8019	no	no	cellular
4	max	95	unknown	single	unknown	yes	102127	yes	yes	unknown



Count the records by group

■ `df.groupby('education').count().show()`

```
1 df.groupby('education').count().show()
```

```
+-----+-----+
|education|count|
+-----+-----+
|   unknown| 1857|
|   tertiary|13301|
|secondary|23202|
|   primary| 6851|
+-----+-----+
```



Count the records by target

■ `df.groupby(target_variable_name).count().show()`

```
1 df.groupby(target_variable_name).count().show()
```

```
+-----+
| y|count|
+-----+
| no|39922|
|yes| 5289|
+-----+
```



Group by Multiple columns

- `df.groupBy(['education',target_variable_name]).count().show()`

```
1 df.groupBy(['education',target_variable_name]).count().show()
```

education	y	count
unknown	no	1605
tertiary	no	11305
secondary	no	20752
unknown	yes	252
primary	no	6260
primary	yes	591
secondary	yes	2450
tertiary	yes	1996



Group by Multiple columns

- `from pyspark.sql.functions import *`
- `df.groupBy(target_variable_name).agg({'balance':'avg', 'age': 'avg'}).show()`

```
1 from pyspark.sql.functions import *
2 df.groupBy(target_variable_name).agg({'balance':'avg', 'age': 'avg'}).show()
```

y	avg(balance)	avg(age)
no	1303.7149691899203	40.83898602274435
yes	1804.2679145396105	41.670069956513515



Cardinality Check

```

1 from pyspark.sql.functions import approxCountDistinct, countDistinct
2
3 def cardinality_calculation(df, cut_off=1):
4     cardinality = df.select(*[approxCountDistinct(c).alias(c) for c in df.columns])
5
6     ## convert to pandas for efficient calculations
7     final_cardinality_df = cardinality.toPandas().transpose()
8     final_cardinality_df.reset_index(inplace=True)
9     final_cardinality_df.rename(columns={0: 'Cardinality'}, inplace=True)
10
11     #select variables with cardinality of 1
12     vars_selected = final_cardinality_df['index'][final_cardinality_df['Cardinality'] <= cut_off]
13
14     return final_cardinality_df, vars_selected
15
16 cardinality_df, cardinality_vars_selected = cardinality_calculation(df)

```



Cardinality Check

1 cardinality_df

	index	Cardinality
0	age	76
1	job	11
2	marital	3
3	education	4
4	default	2
5	balance	7375
6	housing	2



Missing value check

```

1 from pyspark.sql.functions import count, when, isnan, col
2
3 # miss_percentage 80%
4 def missing_calculation(df, miss_percentage=0.80):
5
6     # checks for both NaN and null values
7     missing = df.select(*(count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df.columns))
8     length_df = df.count()
9     ## convert to pandas for efficient calculations
10    final_missing_df = missing.toPandas().transpose()
11    final_missing_df.reset_index(inplace=True)
12    final_missing_df.rename(columns={0: 'missing_count'}, inplace=True)
13    final_missing_df['missing_percentage'] = final_missing_df['missing_count']/length_df
14
15    #select variables with cardinality of 1
16    vars_selected = final_missing_df['index'][final_missing_df['missing_percentage'] >= miss_percentage]
17
18    return final_missing_df, vars_selected

```



Techniques for Converting Strings to Numbers.

- 개별적으로 각 컬럼마다 진행
- OneHotEncoder
- StringIndexer
- Weighted Index

← 라우팅 one Hot Encoding 사용.

→ Index는 정해진 범주에서 숫자 값을 할당하는 방법.



Converting All String Columns to be Number ones.

■ One hot encoder

- 컬럼의 가중치를 고려하지 않음

DATA

but column이 확

Education	Education_Primary	Education_Secondary	Education_Tertiary	Education_Unkown
primary	1	0	0	0
secondary	0	1	0	0
tertiary	0	0	1	0
Unknown	0	0	0	1
Primary	1	0	0	0
secondary	0	1	0	0



Converting All String Columns to be Number ones.

■ StringIndexer

- 순서가 무시되기 때문에 사용할 때 주의

고려됨.

Education	Education_Index
primary	0
secondary	1
tertiary	2
Unknown	3
Primary	0
secondary	1



변환 방법

- #c String 변수를 가진 특징(컬럼)을 선택
- #j 특정 변환기를 선택된 컬럼에 적용
- #r Assemble Features into Feature Vectors
- #s Apply Scaler to Feature Vectors Assembled



#1 String 변수를 가진 특징(컬럼)을 선택

```

1 def variable_type(df):
2
3     vars_list = df.dtypes
4     char_vars = []
5     num_vars = []
6     for i in vars_list:
7         if i[1] in ('string'):
8             char_vars.append(i[0])
9         else:
10            num_vars.append(i[0])
11
12     return char_vars, num_vars

```

숫자, 문자 여부 판별.

```
[35] 1 char_vars, num_vars = variable_type(df)
```

```
[36] 1 char_vars
```

```
[37] 1 num_vars
```



#2 특정 변환기를 선택된 컬럼에 적용

```

38] 1 from pyspark.ml.feature import StringIndexer
    2 from pyspark.ml import Pipeline
    3
    4 def category_to_index(df, char_vars):
    5     char_df = df.select(char_vars)
    6     indexers = [StringIndexer(inputCol=c, outputCol=c+"_index", handleInvalid="keep") for c in char_df.columns]
    7     pipeline = Pipeline(stages=indexers)
    8     char_labels = pipeline.fit(char_df)
    9     df = char_labels.transform(df)
   10     return df, char_labels

39] 1 df, char_labels = category_to_index(df, char_vars)

▶ 1 df = df.select([c for c in df.columns if c not in char_vars])

41] 1 def rename_columns(df, char_vars):
    2     mapping = dict(zip([i + '_index' for i in char_vars], char_vars))
    3     df = df.select([col(c).alias(mapping.get(c, c)) for c in df.columns])
    4     return df

```



#2 특정 변환기를 선택된 컬럼에 적용

```
[42] 1 df = rename_columns(df, char_vars)
```

```
[44] 1 df.dtypes
```

```
[46] 1 df.describe().toPandas()
```



#3 Assemble Features into Feature Vectors

```

1 from pyspark.ml.feature import VectorAssembler
2
3 #assemble individual columns to one column - 'features'
4 def assemble_vectors(df, features_list, target_variable_name):
5     stages = []
6     #assemble vectors
7     assembler = VectorAssembler(inputCols=features_list, outputCol='features')
8     stages = [assembler]
9     #select all the columns + target + newly created 'features' column
10    selectedCols = [target_variable_name, 'features'] + features_list
11    #use pipeline to process sequentially
12    pipeline = Pipeline(stages=stages)
13    #assembler model
14    assembleModel = pipeline.fit(df)
15    #apply assembler model on data
16    df = assembleModel.transform(df).select(selectedCols)
17
18    return df

```

```

[49] 1 #exclude target variable and select all other feature vectors
2     features_list = df.columns
3     #features_list = char_vars #this option is used only for ChiSqSelector
4     features_list.remove(target_variable_name)

```

```

[50] 1 features_list

```



#3 Assemble Features into Feature Vectors

```

1 # apply the function on our dataframe
2 df = assemble_vectors(df, features_list, target_variable_name)

```

```

[52] 1 df.show()

```

```

[61] 1 df.schema["features"].metadata["ml_attr"]["attrs"]

```

```

[54] 1 import pandas as pd
2     for k, v in df.schema["features"].metadata["ml_attr"]["attrs"].items():
3         features_df = pd.DataFrame(v)

```

```

[55] 1 features_df

```

```
df.describe().toPandas()
```



#3 Assemble Features into Feature Vectors

```
[34] 1 # apply the function on our dataframe
      2 df = assemble_vectors(df, features_list, target_variable_name)
```

```
1 df.show()
```

	y	features	age	balance	day	duration	campaign	pdays	previous	job	marital	education	default	housing	loan	contact	month	poutcome
0.0	16	[0,1,2,3,4,5,...]	58	2143	5	261	1	-1	0	1.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	44	29	5	151	1	-1	0	2.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	33	2	5	76	1	-1	0	7.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	47	1506	5	92	1	-1	0	0.0	0.0	3.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	33	[0,1,0,5,0,198,...]	33	1	5	198	1	-1	0	11.0	1.0	3.0	0.0	1.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	35	231	5	139	1	-1	0	1.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	28	[0,447,0,5,0,2,...]	28	447	5	217	1	-1	0	1.0	1.0	1.0	0.0	0.0	1.0	1.0	0.0	0.0
0.0	42	[0,2,0,5,0,380,...]	42	2	5	380	1	-1	0	7.0	2.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	58	121	5	50	1	-1	0	5.0	0.0	2.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	43	593	5	55	1	-1	0	2.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	41	270	5	222	1	-1	0	3.0	2.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	29	390	5	137	1	-1	0	3.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	53	6	5	517	1	-1	0	2.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	58	71	5	71	1	-1	0	2.0	0.0	3.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	57	162	5	174	1	-1	0	4.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	51	229	5	353	1	-1	0	5.0	0.0	2.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	45	[0,13,0,5,0,98,...]	45	13	5	98	1	-1	0	3.0	1.0	3.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	57	52	5	38	1	-1	0	0.0	0.0	2.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,1,2,3,4,5,...]	60	60	5	219	1	-1	0	5.0	0.0	2.0	0.0	0.0	0.0	1.0	0.0	0.0
0.0	16	[0,2,3,4,5,7,...]	33	0	5	54	1	-1	0	4.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0

only showing top 20 rows

```
df.describe().toPandas()
```



#3 Assemble Features into Feature Vectors

```
[36] 1 df.schema
```

```
StructType(List(StructField(y,DoubleType,false),StructField(features,VectorUDT,true),Structf
```

```
1 df.schema["features"].metadata["ml_attr"]["attrs"]
```

```
1 import pandas as pd
2 features_df = None
3 for k, v in df.schema["features"].metadata["ml_attr"]["attrs"].items():
4     if features_df is None:
5         features_df = pd.DataFrame(v)
6     else:
7         features_df = pd.concat([features_df, pd.DataFrame(v)], axis=0)
```

```
[118] 1 features_df = features_df.loc[:, ['idx', 'name']]
      2 features_df
```

	idx	name
0	0	age
1	1	balance
2	2	day
3	3	duration
4	4	campaign
5	5	pdays
6	6	previous
0	7	job
1	8	marital
2	9	education
3	10	default
4	11	housing
5	12	loan
6	13	contact
7	14	month
8	15	poutcome



#4 Apply Scaler to Feature Vectors Assembled

■ StandardScaler

NR

Min-Max

Standardization:

$$z = \frac{x - \mu}{\sigma}$$

with mean:

$$\mu = \frac{1}{N} \sum_{i=1}^N (x_i)$$

and standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$



#4 Apply Scaler to Feature Vectors Assembled

```

1 from pyspark.ml.feature import VectorAssembler
2 from pyspark.ml.feature import StandardScaler
3
4 #assemble and scale individual columns to one column - 'features2'
5 def scaled_assemble_vectors(df, features_list, target_variable_name):
6     stages = []
7     #assemble vectors
8     assembler = VectorAssembler(inputCols=features_list, outputCol='assembled_features')
9     scaler = StandardScaler(inputCol=assembler.getOutputCol(), outputCol='features2')
10    stages = [assembler, scaler]
11    #select all the columns + target + newly created 'features' column
12    selectedCols = [target_variable_name, 'features2'] + features_list
13    #use pipeline to process sequentially
14    pipeline = Pipeline(stages=stages)
15    #assembler model
16    scaleAssembleModel = pipeline.fit(df)
17    #apply assembler model on data
18    df = scaleAssembleModel.transform(df).select(selectedCols)
19    return df

```



Without Target

- ★ Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- Model-based Feature Selection



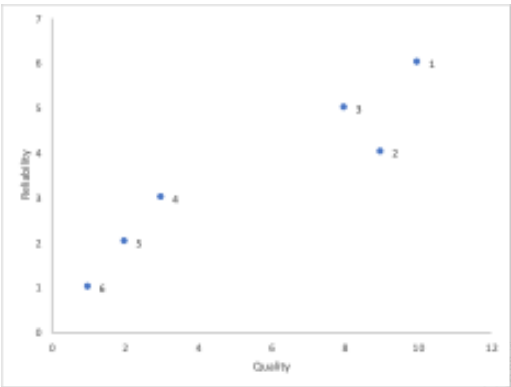
Principal Component Analysis

- 대규모 데이터셋에서 패턴을 식별
 - 대부분의 정보를 포함하는 변수들을 확인
 - 데이터셋에 대한 선형 표현



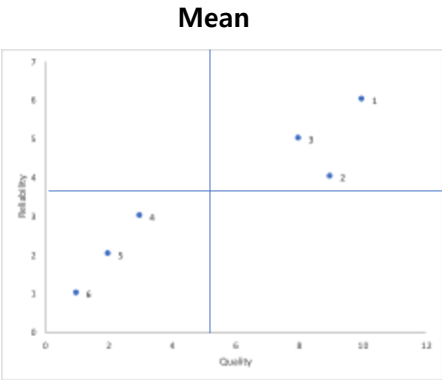
Example: Sample Data

참가자 ID	Quality	Realiability
1	10	6
2	9	4
3	8	5
4	3	3
5	2	2
6	1	1



Example: Sample Data

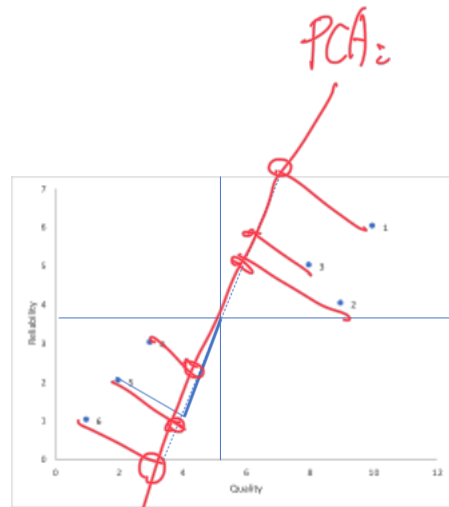
참가자 ID	Quality	Realiability
1	10	6
2	9	4
3	8	5
4	3	3
5	2	2
6	1	1



Example: Sample Data

참가자 ID	Quality	Reliability
1	10	6
2	9	4
3	8	5
4	3	3
5	2	2
6	1	1

특정 Feature의 분산을 최대한으로 유지하는
선형 함수를 PC1



주변값에 걸리지 않고 고차원
공간 DATA

PCA (3/16=1/5)

```
1 from pyspark.ml.feature import PCA
2 from pyspark.ml.linalg import Vectors
3
4 pca = PCA(k=3, inputCol="features", outputCol="pcaFeatures")
5 model = pca.fit(df)
6
7 result = model.transform(df).select("pcaFeatures")
8 result.show(truncate=False)
```

```
+-----+
|pcaFeatures|
+-----+
[[-2143.495364773581,-257.0420740676505,1.2449712753048212]|
[[-29.29226175164628,-150.92888640669165,1.0493026899279903]|
[[-2.150889773941882,-75.98191250436612,1.0760651955381237]|
[[-1506.1823305908117,-89.21547154672699,1.2683999536976271]|
[[-1.3750908349448236,-197.9835617549482,0.9892427009816414]|
[[-231.26679712386618,-138.56034919503713,1.0576960891396285]|
[[-447.4072782675641,-216.1854138839743,1.0066666548031231]|
[[-2.712326444744223,-379.97858281643545,0.8751504698368189]|
[[-121.11144848214049,-49.75636025970778,1.172585359192404]|
[[-593.1146061641075,-53.89364261832033,1.1817022732262128]|
[[-270.4212341323417,-221.48699661827914,1.0170824073205755]|
[[-390.26081676654087,-136.27016909339133,1.0625336662699843]|
[[-6.967616100129188,-516.9674558248696,0.8003545768071488]|
[[-71.15006476185233,-70.84798774752198,1.1499619332808761]|
[[-162.33874829001974,-173.68095250690791,1.0823345183248174]|
[[-229.66539702564629,-352.5590565974901,0.9435034899633544]|
[[-13.195336090120463,-97.9581534500072,1.090278500900193]|
[[-52.08909854893981,-37.88319180402837,1.1682296517559208]|
[[-60.422642726801485,-218.8673212338794,1.0462504939058524]|
[[-0.11044203745873847,-53.98554342962398,1.0903138050958234]|
+-----+
```



Loading scores for PCs

1 model.pc.toArray()

```
array([[ -3.41021399e-04,  2.79524640e-04,  2.58353293e-03],
       [-9.99998245e-01,  1.83654726e-03,  1.13892524e-04],
       [-1.22934480e-05,  9.79995613e-04,  7.79347982e-03],
       [-1.83671689e-03, -9.99996986e-01, -7.36955549e-04],
       [ 1.48468991e-05,  1.01391994e-03,  2.75121381e-03],
       [-1.13085547e-04,  7.49207153e-04, -9.99889046e-01],
       [-1.26153895e-05, -6.36100089e-06, -1.04654388e-02],
       [-1.78789640e-05, -4.11817349e-05,  5.51411389e-04],
       [ 6.41085932e-06, -5.23364803e-05, -1.45349520e-04],
       [-1.11185424e-05,  1.30366514e-05,  2.01500982e-04],
       [ 2.91665702e-06,  4.42643869e-06,  3.95562163e-05],
       [-1.12221341e-05,  1.26153926e-05,  6.17569266e-04],
       [ 1.01623400e-05,  1.50687571e-05,  8.23933054e-05],
       [-5.68377754e-07,  6.95393403e-05,  1.03951369e-03],
       [-7.60886236e-05, -1.16754927e-04, -3.24662847e-03],
       [-8.55162111e-06, -6.01853226e-05, -4.94522998e-03]])
```

DATA

Variable	PC1	PC2	PC3
age	-0.0003	0.0003	0.0026
balance	-1.0000	0.0018	0.0001
day	0.0000	0.0010	0.0078
duration	-0.0018	-1.0000	-0.0007
campaign	0.0000	0.0010	0.0028
pdays	-0.0001	0.0007	-0.9999
previous	0.0000	0.0000	-0.0105
job	0.0000	0.0000	0.0006
marital	0.0000	-0.0001	-0.0001
education	0.0000	0.0000	0.0002
default	0.0000	0.0000	0.0000
housing	0.0000	0.0000	0.0006
loan	0.0000	0.0000	0.0001
contact	0.0000	0.0001	0.0010
month	-0.0001	-0.0001	-0.0032
poutcome	0.0000	-0.0001	-0.0049

컴퓨터AI공학부

61

동아대학교

Loading scores for PCs

1 model.pc.toArray()

```
array([[ -3.41021399e-04,  2.79524640e-04,  2.58353293e-03],
       [-9.99998245e-01,  1.83654726e-03,  1.13892524e-04],
       [-1.22934480e-05,  9.79995613e-04,  7.79347982e-03],
       [-1.83671689e-03, -9.99996986e-01, -7.36955549e-04],
       [ 1.48468991e-05,  1.01391994e-03,  2.75121381e-03],
       [-1.13085547e-04,  7.49207153e-04, -9.99889046e-01],
       [-1.26153895e-05, -6.36100089e-06, -1.04654388e-02],
       [-1.78789640e-05, -4.11817349e-05,  5.51411389e-04],
       [ 6.41085932e-06, -5.23364803e-05, -1.45349520e-04],
       [-1.11185424e-05,  1.30366514e-05,  2.01500982e-04],
       [ 2.91665702e-06,  4.42643869e-06,  3.95562163e-05],
       [-1.12221341e-05,  1.26153926e-05,  6.17569266e-04],
       [ 1.01623400e-05,  1.50687571e-05,  8.23933054e-05],
       [-5.68377754e-07,  6.95393403e-05,  1.03951369e-03],
       [-7.60886236e-05, -1.16754927e-04, -3.24662847e-03],
       [-8.55162111e-06, -6.01853226e-05, -4.94522998e-03]])
```

해상력 향상

Variable	PC1	PC2	PC3
age	-0.0003	0.0003	0.0026
balance	✓ -1.0000	0.0018	0.0001
day	0.0000	0.0010	0.0078
duration	-0.0018	✓ -1.0000	-0.0007
campaign	0.0000	0.0010	0.0028
pdays	-0.0001	0.0007	✓ -0.9999
previous	0.0000	0.0000	-0.0105
job	0.0000	0.0000	0.0006
marital	0.0000	-0.0001	-0.0001
education	0.0000	0.0000	0.0002
default	0.0000	0.0000	0.0000
housing	0.0000	0.0000	0.0006
loan	0.0000	0.0000	0.0001
contact	0.0000	0.0001	0.0010
month	-0.0001	-0.0001	-0.0032
poutcome	0.0000	-0.0001	-0.0049

1 model.explainedVariance

```
DenseVector([0.9918, 0.0071, 0.0011])
```

컴퓨터AI공학부

62

동아대학교

PCA with StandardScaler

```
1 pca = PCA(k=3, inputCol="features2", outputCol="pcaFeatures")
2 model = pca.fit(df)
3
4 result = model.transform(df).select("pcaFeatures")
5 result.show(truncate=False)
```

```
[56] 1 model.explainedVariance
```

```
DenseVector([0.1434, 0.0987, 0.0787])
```

```
pcaFeatures
|
|-----|
| [-0.6567808835228072, 4.3121948711553015, -4.3937648469809] |
| [-0.590553920179786, 2.7430023739115277, -2.0129736282025625] |
| [-0.7923309293106022, 2.428517822202711, -1.412875839724541] |
| [-0.7779852947451491, 4.298302632066033, -5.322505502928547] |
| [-0.5280039534401401, 6.559141503669079, -1.4081500208397828] |
| [-0.7028277580737647, 2.7093407342344897, -3.226355647066135] |
| [-0.8104872327792136, 1.429021001344571, -1.619935441334062] |
| [-0.975873535487279, 2.862044859015957, 0.4335559756415678] |
| [-0.7126916038562845, 5.175637716397523, -4.531453571469944] |
| [-0.5939132026124012, 2.7467852504302592, -2.0709944257197654] |
| [-0.48367202786573626, 2.584502296306621, -0.8133472106594244] |
| [-0.5574665926506135, 2.079820030407318, -1.1695266299980636] |
| [-0.6052039081600511, 3.4943812853193674, -3.06065582479699] |
| [-0.7917889584107352, 5.104448480115379, -5.533708763513019] |
| [-0.634131428409618, 4.061248668356405, -3.2668691358886577] |
| [-0.6515302947285277, 4.807107975840059, -3.9602262772305346] |
| [-0.6913724174639284, 4.317335145410398, -3.8524349637084288] |
| [-0.7928529953941135, 4.2455389078940895, -5.143335647201441] |
| [-0.6860141754953244, 5.301081379539062, -4.4986208881179595] |
| [-0.6396798564710514, 2.621609196420631, -2.1560847881880587] |
|
+-----+
only showing top 20 rows
```



Singular Value Decomposition(특이값 분해)

Singular value decomposition (SVD) factorizes a matrix into three matrices: U , Σ , and V such that

$$A = U\Sigma V^T,$$

where

- U is an orthonormal matrix, whose columns are called left singular vectors,
- Σ is a diagonal matrix with non-negative diagonals in descending order, whose diagonals are called singular values,
- V is an orthonormal matrix, whose columns are called right singular vectors.

For large matrices, usually we don't need the complete factorization but only the top singular values and its associated singular vectors. This can save storage, de-noise and recover the low-rank structure of the matrix.

If we keep the top k singular values, then the dimensions of the resulting low-rank matrix will be:

- $U: m \times k$,
- $\Sigma: k \times k$,
- $V: n \times k$.

→ 특이값을 사용해 경량화.

$$\text{ex) } U \times V = 100 \times 10 \\ = 1000.$$

$$U \times \Sigma \times V = 100 \times k \\ k \times k \\ 10 \times k$$

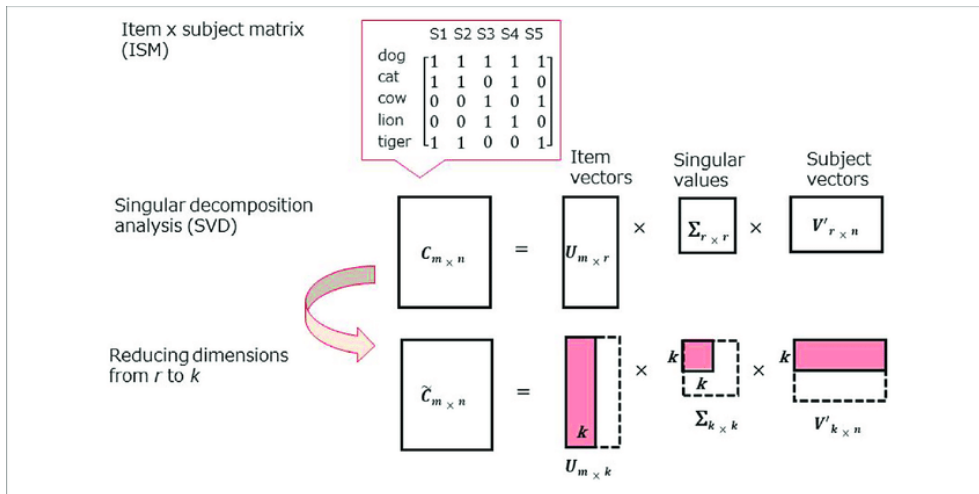
$$\begin{matrix} \text{특이값} \\ 300 \\ 9 \\ 30 \end{matrix} = 339$$

$$\therefore 1000 > 339.$$



Singular Value Decomposition(특이값 분해)

■ 데이터 압축



- Gives a decomposition of any matrix into a product of three matrices:

$$\begin{matrix} n \\ m \end{matrix} \begin{matrix} A \end{matrix} \sim \begin{matrix} r \\ m \end{matrix} \begin{matrix} U \end{matrix} \times \begin{matrix} r \\ r \end{matrix} \begin{matrix} \Sigma \end{matrix} \times \begin{matrix} n \\ r \end{matrix} \begin{matrix} V^T \end{matrix}$$

↓ SVD

- There are strong constraints on the form of each of these matrices
 - Results in a unique decomposition
- From this decomposition, you can choose any number r of intermediate concepts (latent factors) in a way that minimizes the reconstruction error

■ $A = U \Sigma V^T$ - example: Users to Movies

$$\begin{array}{c} \uparrow \\ \text{SciFi} \\ \downarrow \\ \uparrow \\ \text{Romance} \\ \downarrow \end{array}
 \begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array}
 \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix}
 =
 \begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix}
 \times
 \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix}
 \times
 \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\ 0.40 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$

SVD (값재분배)



■ $A = U \Sigma V^T$ - example: Users to Movies

$$\begin{array}{c} \uparrow \\ \text{SciFi} \\ \downarrow \\ \uparrow \\ \text{Romance} \\ \downarrow \end{array}
 \begin{array}{c} \text{Matrix} \\ \text{Alien} \\ \text{Serenity} \\ \text{Casablanca} \\ \text{Amelie} \end{array}
 \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 3 & 3 & 3 & 0 & 0 \\ 4 & 4 & 4 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 \\ 0 & 2 & 0 & 4 & 4 \\ 0 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & 2 & 2 \end{bmatrix}
 =
 \begin{array}{c} \text{SciFi-concept} \\ \downarrow \\ \text{Romance-concept} \end{array}
 \begin{bmatrix} 0.13 & 0.02 & -0.01 \\ 0.41 & 0.07 & -0.03 \\ 0.55 & 0.09 & -0.04 \\ 0.68 & 0.11 & -0.05 \\ 0.15 & -0.59 & 0.65 \\ 0.07 & -0.73 & -0.67 \\ 0.07 & -0.29 & 0.32 \end{bmatrix}
 \times
 \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix}
 \times
 \begin{bmatrix} 0.56 & 0.59 & 0.56 & 0.09 & 0.09 \\ 0.12 & -0.02 & 0.12 & -0.69 & -0.69 \\ 0.40 & -0.80 & 0.40 & 0.09 & 0.09 \end{bmatrix}$$



SVD(특이값 분해)

▶

1 U.rows.collect()

⌘

DenseVector([-0.0002, 0.0034, 0.0009, 0.0051, 0.0067]),
DenseVector([-0.0008, 0.0028, 0.0008, 0.0022, 0.0024]),
DenseVector([-0.0, 0.0076, 0.0021, -0.0011, 0.003]),
DenseVector([-0.0008, 0.0035, 0.001, 0.0022, 0.0033]),
DenseVector([-0.0021, 0.0, 0.0003, 0.0059, 0.0053]),
DenseVector([-0.0029, 0.0042, 0.0016, -0.0002, 0.0025]),
DenseVector([-0.0026, 0.001, 0.0007, 0.0031, 0.0032]),
DenseVector([-0.0002, 0.005, 0.0014, 0.0003, 0.0021]),
DenseVector([-0.0003, 0.0056, 0.0015, 0.003, 0.0064]),
DenseVector([-0.0003, 0.0016, 0.0004, 0.0037, 0.0028]),
DenseVector([-0.0002, 0.0005, 0.0001, 0.0053, 0.0036]),
DenseVector([-0.0001, 0.0044, 0.0011, 0.0029, 0.0049]),
DenseVector([-0.0009, 0.0, 0.0001, 0.0059, 0.0046]),
DenseVector([-0.0004, 0.0115, 0.0032, -0.0053, 0.002]),
DenseVector([-0.0007, 0.0029, 0.0008, 0.0059, 0.0075]),
DenseVector([0.0, 0.0021, 0.0005, 0.0042, 0.0038]),
DenseVector([-0.0002, 0.0007, 0.0001, 0.0053, 0.004]),
DenseVector([-0.0005, 0.021, 0.0059, -0.0121, 0.0038]),
DenseVector([-0.0013, 0.0012, 0.0004, 0.0069, 0.0075]),
DenseVector([-0.0003, 0.0024, 0.0006, 0.0046, 0.005]),
DenseVector([-0.0012, 0.0024, 0.0008, 0.0047, 0.0058]),
DenseVector([-0.0002, 0.0037, 0.001, 0.0038, 0.0055]),

[62] 1 s

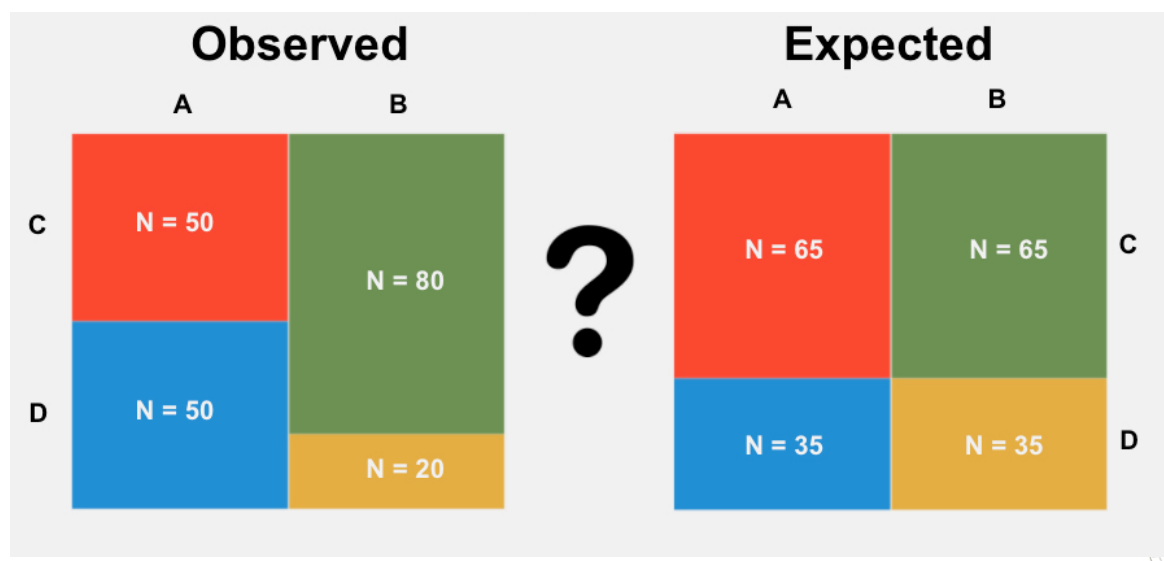
DenseVector([709656.9403, 74266.7144, 22052.8227, 6458.1862, 1846.0544])

[66] 1 V

DenseMatrix(16, 5, [-0.0053, -0.9994, -0.002, -0.0335, -0.0003, -0.005, -0.0001, -0.0003,



ChiSq Selector(카이제곱 선택)



ChiSq Selector(카이제곱 선택)

- $\chi^2 = \sum (\text{관측값} - \text{기댓값})^2 / \text{기댓값}$
 - 적합도 검정(Goodness of fit)
- Null Hypothesis vs. Alternative Hypothesis

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

χ^2 : 카이제곱, O: 관측된 데이터, E: 기대되는 데이터



ChiSq Selector(카이제곱 선택)

- 생존자 비교: 성별과 생존자 간에 관계가 있는가?
 - 여성 생존자: 67.8% (120/290)
 - 남성 생존자: 21% (109/290)
- Null Hypo.: 성별과 생존자간에는 관계없다

Gender		Survived		Total
		No	Yes	
	Female	81	231	312
	Male	468	109	577
	Total	549	340	889

Titanic example: Gender survived



ChiSq Selector(카이제곱 선택)

■ 기대값 계산

Gender	Survived		Expected Value
Female	No	$(312 \times 549) / 889$	192.67
Female	Yes	$(312 \times 340) / 889$	119.32
Male	No	$(577 \times 549) / 889$	356.32
Male	Yes	$(577 \times 340) / 889$	220.67

		Survived		
		No	Yes	Total
Gender	Female	81	231	312
	Male	468	109	577
	Total	549	340	889

Titanic example: Gender survived



ChiSq Selector(카이제곱 선택)

■ 카이제곱 값

Gender	Survived	Expected Value	Observed	Chi-Square
Female	No	192.67	81	64.72
Female	Yes	119.32	231	104.51
Male	No	356.32	468	34.99
Male	Yes	220.67	109	56.51
				260.75

		Survived		
		No	Yes	Total
Gender	Female	81	231	312
	Male	468	109	577
	Total	549	340	889

Titanic example: Gender survived



ChiSq Selector(카이제곱 선택)

■ 카이제곱 값

Gender	Survived	Expected Value	Observed	Chi-Square
Female	No	192.67	81	64.72
Female	Yes	119.32	231	104.51
Male	No	356.32	468	34.99
Male	Yes	220.67	109	56.51
				260.75

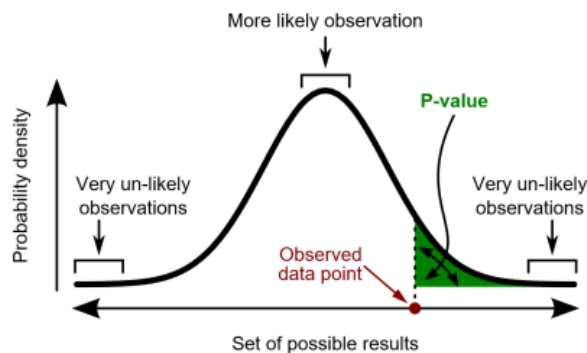
		Survived		
Gender		No	Yes	Total
	Female	81	231	312
	Male	468	109	577
	Total	549	340	889

Titanic example: Gender survived



P-value(유의 확률)

- 테스트한 결과를 얻을 확률
- Null hypothesis testing 참이라고 할 때,
 - P-value 가 0.05 이하 이라면, null hypothesis is (기각)rejected



P-value 선택

■ o에 가까울 수록, 주어진 셀간 의미가 있다

● df는 (rows-1) * (cols-1)

아는 표
사용 가능.

df	0.995	0.99	0.975	0.95	0.90	0.10	0.05	0.025	0.01	0.005
1	---	---	0.001	0.004	0.016	2.706	3.841	5.024	6.635	7.879
2	0.010	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210	10.597
3	0.072	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345	12.838
4	0.207	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277	14.860
5	0.412	0.554	0.831	1.145	1.610	9.236	11.070	12.833	15.086	16.750
6	0.676	0.872	1.237	1.635	2.204	10.645	12.592	14.449	16.812	18.548
7	0.989	1.239	1.690	2.167	2.833	12.017	14.067	16.013	18.475	20.278
8	1.344	1.646	2.180	2.733	3.490	13.362	15.507	17.535	20.090	21.955
9	1.735	2.088	2.700	3.325	4.168	14.684	16.919	19.023	21.666	23.589

신뢰수준 95%

→ p-value (2.2243)

<https://people.richland.edu/james/lecture/m170/tbl-chi.html>



Tardiness Check

귀무가설: 매 요일마다 동등하게 지각인원이 발생한다
대안가설: 매 요일마다 동등하게 지각인원이 발생하지 않는다.
Significance level(검정수준) 5%

Week of day /	Monday	Tuesday	Wednesday	Thursday	Friday
Observed	9	15	8	13	8
Expected	10	10	10	10	10

col = 5
row = 2

∴ df = (row-1) * (col-1)
= 4

카이제곱 = $\frac{1^2 + 5^2 + 2^2 + 3^2 + 2^2}{10} = 4.3 \Rightarrow 9.490$
p-Value.



ChiSq Selector

```

1 from pyspark.ml.feature import ChiSqSelector
2 from pyspark.ml.linalg import Vectors
3
4 selector = ChiSqSelector(numTopFeatures=6, featuresCol="features",
5                           outputCol="selectedFeatures", labelCol="y")
6
7 chi_selector = selector.fit(df)
8
9 result = chi_selector.transform(df)
10
11 print("ChiSqSelector output with top %d features selected" % selector.getNumTopFeatures())
12 print("Selected Indices: ", chi_selector.selectedFeatures)
13 result.show()

```

```

+-----+
| selectedFeatures |
+-----+
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|[33.0,1.0,5.0,198,...] |
|(6,[0,1,2,3,4,5],...) |
|[28.0,447.0,5.0,2,...] |
|[42.0,2.0,5.0,380,...] |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|[45.0,13.0,5.0,98,...] |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,1,2,3,4,5],...) |
|(6,[0,2,3,4,5],[3,...] |
+-----+

```



ChiSq Selector

```

1 features_df['chisq_importance'] = features_df['idx'].apply(lambda x: 1 if x in chi_selector.selectedFeatures else 0)

```

```

1 features_df

```

	idx	name	chisq_importance
0	0	age	1
1	1	balance	1
2	2	day	1
3	3	duration	1
4	4	campaign	1
5	5	pdays	1
6	6	previous	0

0	7	job	0
1	8	marital	0
2	9	education	0
3	10	default	0
4	11	housing	0
5	12	loan	0
6	13	contact	0
7	14	month	0
8	15	poutcome	0



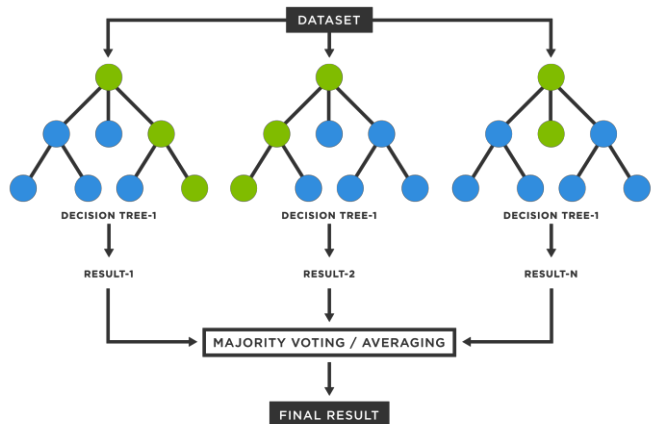
Model-based Feature Selection

- Make a computer to distinguish between dogs and cats
 - In Spark, Tree-based methods is provided by default
 - Decision Tree
 - Random Forest
 - Gradient Boosting



Random Forest

- Supervised machine learning algorithm



Random Forest

```
[124] 1 from pyspark.ml.classification import RandomForestClassifier
      2
      3 rf = RandomForestClassifier(featuresCol='features', labelCol=target_variable_name)
      4 rf_model = rf.fit(df)
      5 rf_model.featureImportances
```

SparseVector(16, {0: 0.0174, 1: 0.0025, 2: 0.0048, 3: 0.4374, 4: 0.0026, 5: 0.0302, 6: 0.041, 7: 0.0053, 8: 0.0035, 9: 0.0...

```
[125] 1 #temporary output rf_output
      2 rf_output = rf_model.featureImportances
      3 features_df['Importance'] = features_df['idx'].apply(lambda x: rf_output[x] if x in rf_output.indices else 0)
```

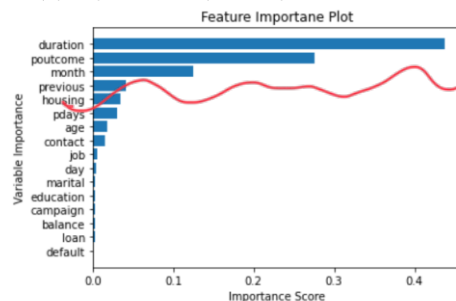
```
[126] 1 #sort values based on descending importance feature
      2 features_df.sort_values("Importance", ascending=False, inplace=True)
```



Random Forest

```
[128] 1 import matplotlib.pyplot as plt
      2
      3 features_df.sort_values("Importance", ascending=True, inplace=True)
      4 plt.barh(features_df['name'], features_df['Importance'])
      5 plt.title("Feature Importane Plot")
      6 plt.xlabel("Importance Score")
      7 plt.ylabel("Variable Importance")
```

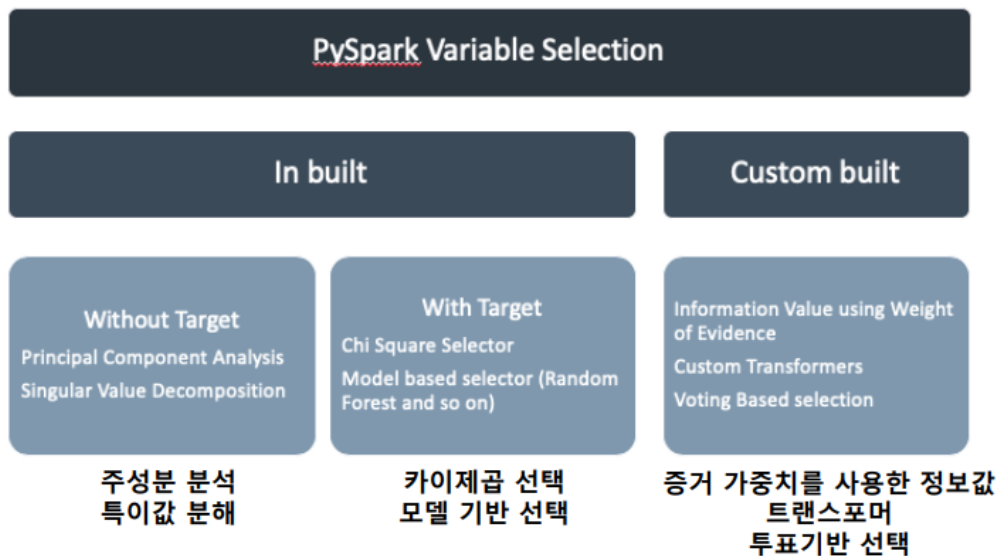
Text(0, 0.5, 'Variable Importance')



시간



PySpark 기반 변수 선택 기법



Custom-built Variable Selection

Contents

- Information Value using Weight of Evidence
- Custom Transformers
- Voting-based Selection



Information Value(IV) using Weight of Evidence(WoE)

- Variable transformation과 selection에서 강력한 도구
- 신용점수 측정에 주로 사용됨
 - 만약 고객이 지불 불이행(default on a payment)을 할지 예측
 - Events와 Non Events로 분류
 - Events: 지불 불이행을 한 고객
 - Non Events: 지불 불이행을 하지 않은 고객

$$WOE = \ln\left(\frac{\text{Event\%}}{\text{Non Event\%}}\right)$$

$$IV = \sum (\text{Event\%} - \text{Non Event\%}) * \ln\left(\frac{\text{Event\%}}{\text{Non Event\%}}\right)$$

WOE



Information Value(IV) using Weight of Evidence(WoE)

- Handles Missing values
- Handles Outliers
- Transformation은 분포는 로그값에 기반하기 때문에,
- 로그 기반의 회귀 값들과 정렬 가능함
- 불필요한 변수를 요구하지 않음
- 적절한 bin 테크닉을 사용하여, 독립과 종속 변수간에
- monotonic 관계를 만들어 낼 수 있음



Information Value(IV) using Weight of Evidence(WoE)

$$WOE = \ln\left(\frac{\text{Event}\%}{\text{Non Event}\%}\right) \quad IV = \sum (\text{Event}\% - \text{Non Event}\%) * \ln\left(\frac{\text{Event}\%}{\text{Non Event}\%}\right)$$

$$IV = \sum (\text{Event}\% - \text{Non Event}\%) * (WOE)$$

Variable Name	Min. Value	Max. Value	Count	# Event	# Non Event	Event%	Non event%	WOE	Event% - Non event%	IV
Age	10	20	1200	150	1050	28.3%	19.0%	0.3992	9.3%	0.03718
Age	21	30	900	120	780	22.6%	14.1%	0.4733	8.5%	0.04040
Age	31	40	1090	110	980	20.8%	17.7%	0.1580	3.0%	0.00479
Age	41	50	1460	100	1360	18.9%	24.6%	-0.2650	-5.7%	0.01517
Age	50	inf	1410	50	1360	9.4%	24.6%	-0.9582	-15.2%	0.14525
Total			6060	530	5530					0.24279



Information Value(IV) using Weight of Evidence(WoE)

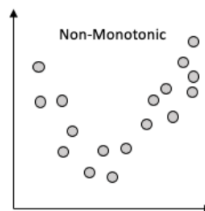
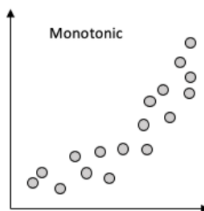
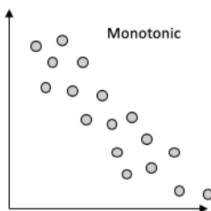
$$IV = \sum (\text{Event\%} - \text{Non Event\%}) * (\text{WOE})$$

Information Value (IV)	Predictive Power
< 0.02	useless for prediction
0.02 to 0.1	weak predictor
0.1 to 0.3	medium predictor
0.3 to 0.5	strong predictor
> 0.5	suspicious or too good to be true

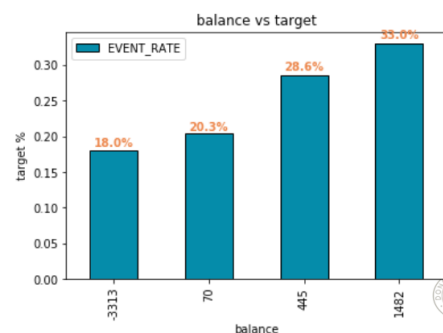
Variable Name	Min. Value	Max. Value	Count	# Event	# Non Event	Event%	Non event%	WOE	Event% - Non event%	IV
Age	10	20	1200	150	1050	28.3%	19.0%	0.3992	9.3%	0.03718
Age	21	30	900	120	780	22.6%	14.1%	0.4733	8.5%	0.04040
Age	31	40	1090	110	980	20.8%	17.7%	0.1580	3.0%	0.00479
Age	41	50	1460	100	1360	18.9%	24.6%	-0.2650	-5.7%	0.01517
Age	50	inf	1410	50	1360	9.4%	24.6%	-0.9582	-15.2%	0.14525
Total			6060	530	5530					0.24279



Monotonicity



두 특징 간에 증가(Increase) 혹은 감소(Decrease)의 행위 관계를 보여주는 것



Spearman Correlation

■ 두 데이터 간의 상관관계

- 방향과 연관성을 계산할 수 있음
- d 는 두 데이터간의 차이

Mathematics (mark)	Physics (mark)	Rank for Mathematics	Rank for Physics	d_i	d_i^2
48	56	7	7	0	0
75	89	3	2	1	1
53	65	6	6	0	0
68	72	5	5	0	0
92	74	1	4	-3	9
88	81	2	3	-1	1
74	99	4	1	3	9
Total					20



Spearman Correlation

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

=0.64

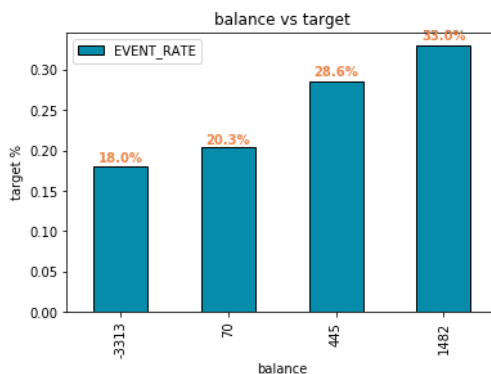
Absolute ρ value	Relationship
0.00 to 0.19	very weak
0.20 to 0.39	weak
0.40 to 0.59	moderate
0.60 to 0.79	strong
0.80 to 1.0	very strong

Mathematics (mark)	Physics (mark)	Rank for Mathematics	Rank for Physics	d_i (정차)	d_i^2 (정차제곱)
48	56	7	7	0	0
75	89	3	2	1	1
53	65	6	6	0	0
68	72	5	5	0	0
92	74	1	4	-3	9
88	81	2	3	-1	1
74	99	4	1	3	9
Total (50d)					20



Monotonicity

- Bin을 효과적으로 설계하는 것이 중요



멜버른 집값 데이터

Melbourne Housing Snapshot
Snapshot of Tony Pino's Melbourne Housing Dataset

DanB • updated 4 years ago (Version 5)

Data Tasks (1) Code (4,494) Discussion (8) Activity Metadata

Download (2 MB) New Notebook

Usability 7.1 License CC BY-NC-SA 4.0 Tags social science, real estate, demographics, housing, australia

Description

Context

Melbourne real estate is BOOMING. Can you find the insight or predict the next big trend to become a real estate mogul... or even harder, to snap up a reasonably priced 2-bedroom unit?

Content

This is a snapshot of a dataset created by Tony Pino.

It was scraped from publicly available results posted every week from Domain.com.au. He cleaned it well, and now it's up to you to make data analysis magic. The dataset includes Address, Type of Real estate, Suburb, Method of Selling, Rooms, Price, Real Estate Agent, Date of Sale and distance from C.B.D.

Notes on Specific Variables



멜버른 집값 데이터

■ 집의 유형(Type)과 관계된 특징을 추출하는 것이 목표

Notes on Specific Variables

Rooms: Number of rooms

Price: Price in dollars

Method: S - property sold; SP - property sold prior; PI - property passed in; PN - sold prior not disclosed; SN - sold not disclosed; NB - no bid; VB - vendor bid; W - withdrawn prior to auction; SA - sold after auction; SS - sold after auction price not disclosed. N/A - price or highest bid not available.

Type: br - bedroom(s); h - house, cottage, villa, semi, terrace; u - unit, duplex; t - townhouse; dev site - development site; o res - other residential.

SellerG: Real Estate Agent

Date: Date sold

Distance: Distance from CBD

Regionname: General Region (West, North West, North, North east ...etc)

Propertycount: Number of properties that exist in the suburb.

Bedroom2 : Scraped # of Bedrooms (from different source)

Bathroom: Number of Bathrooms

Car: Number of carspots

Landsize: Land Size

BuildingArea: Building Size

CouncilArea: Governing council for the area



Colab에서 Notebook 생성

■ Spark setup은 동일하게

```
1 !wget -q https://raw.githubusercontent.com/esabunor/MLWorkspace/master/melb_data.csv
```

Load Dataset

```
[15] 1 from pyspark.sql import functions as F
2
3 filename = "melb_data.csv"
4 target_variable_name = "type"
5 df = spark.read.csv(filename, header=True, inferSchema=True, sep=',')
6 # type h - house, cottage, villa, semi, terrace
7 #   other residential.
8 df = df.withColumn(target_variable_name, F.when(df[target_variable_name] == 'h', 0).otherwise(1))
9
10 df.show()
```



데이터 확인

■ 집의 유형

- o: house, villa의 주거형태
- o: 다른 주거 형태

■ 데이터 유형

- 가격
- 욕실/방

```
[16] | df.groupby(target_variable_name).count().show()
```

```
+-----+
|type|count|
+-----+
| 1| 4131|
| 0| 9449|
+-----+
```

```
| df.dtypes
```

```
[('Suburb', 'string'),
 ('Address', 'string'),
 ('Rooms', 'int'),
 ('type', 'int'),
 ('Price', 'double'),
 ('Method', 'string'),
 ('SellerG', 'string'),
 ('Date', 'string'),
 ('Distance', 'double'),
 ('Postcode', 'double'),
 ('Bedroom2', 'double'),
 ('Bathroom', 'double'),
 ('Car', 'double'),
 ('Landsize', 'double'),
 ('BuildingArea', 'double'),
 ('YearBuilt', 'double'),
 ('CouncilArea', 'string'),
 ('Latitude', 'double'),
 ('Longitude', 'double'),
 ('Regionname', 'string'),
 ('Propertycount', 'double')]
```



데이터 확인

- 문자형, 숫자형 데이터 분리
- 불필요한 데이터필드 제거

■ 다음사항도 확인

- 카디날리티
- 결측값

```
[18] | 1 #identify variable types and perform some operations
      2 def variable_type(df):
      3
      4     vars_list = df.dtypes
      5     char_vars = []
      6     num_vars = []
      7     for i in vars_list:
      8         if i[1] in ('string'):
      9             char_vars.append(i[0])
      10        else:
      11            num_vars.append(i[0])
      12
      13    return char_vars, num_vars
```

```
| 1 char_vars, num_vars = variable_type(df)
  2
  3 char_vars.remove('Address')
  4 char_vars.remove('SellerG')
  5 char_vars.remove('Date')
  6 char_vars.remove('Suburb')
```

```
[21] | 1 num_vars.remove(target_variable_name)
```

```
[22] | 1 final_vars = char_vars + num_vars
```



라이브러리 Import

- WOE 계산 함수
- Pearson correlation을 위한 Monotonic 함수
- 특징별로 WOE 계산 실행

```
1 # WOE & IV code
2 from pyspark.sql import functions as F
3 import pandas as pd
4 import numpy as np
5 from pyspark.ml.feature import QuantileDiscretizer
6 from pyspark.ml.feature import VectorAssembler
7 import scipy.stats.stats as stats
```



WOE 계산 함수

```
23 1 #default parameters
2 #rho value for spearman correlation. You can adjust this value
3 custom_rho = 1
4 #maximum number of bins to start of with. It will keep decreasing from this number. Adjustable parameter
5 max_bin = 20
6
7 def calculate_woe(count_df, event_df, min_value, max_value, feature):
8
9     # implementing the table structure shown in WOE & IV calculation figure
10    woe_df = pd.merge(left=count_df, right=event_df)
11    woe_df['min_value'] = min_value
12    woe_df['max_value'] = max_value
13    woe_df['non_event'] = woe_df['count'] - woe_df['event']
14    woe_df['event_rate'] = woe_df['event']/woe_df['count']
15    woe_df['nonevent_rate'] = woe_df['non_event']/woe_df['count']
16    woe_df['dist_event'] = woe_df['event']/woe_df['event'].sum()
17    woe_df['dist_nonevent'] = woe_df['non_event']/woe_df['non_event'].sum()
18    woe_df['woe'] = np.log(woe_df['dist_event']/woe_df['dist_nonevent'])
19    woe_df['iv'] = (woe_df['dist_event'] - woe_df['dist_nonevent'])*woe_df['woe']
20    woe_df['varname'] = [feature]* len(woe_df)
21    woe_df = woe_df[['varname', 'min_value', 'max_value', 'count', 'event', 'non_event', 'event_rate', 'nonevent_rate', 'dist_event', 'dist_nonevent', 'woe', 'iv']]
22    woe_df = woe_df.replace([np.inf, -np.inf], 0)
23    woe_df['iv'] = woe_df['iv'].sum()
24    return woe_df
25
```



Pearson correlation을 위한 Monotonic 함수

```

1
2 #monotonic binning function implemented along with spearman correlation
3 def mono_bin(temp_df, feature, target, n = max_bin):
4
5     r = 0
6     while np.abs(r) < custom_rho and n > 1:
7
8         try:
9             #Quantile discretizer cuts data into equal number of observations
10            qds = QuantileDiscretizer(numBuckets=n, inputCol=feature, outputCol='buckets', relativeError=0.01)
11            bucketizer = qds.fit(temp_df)
12            temp_df = bucketizer.transform(temp_df)
13
14            #create corr_df is Python based implementation for efficiency purpose
15            corr_df = temp_df.groupby('buckets').agg({feature:'avg', target:'avg'}).toPandas()
16            corr_df.columns = ['buckets', feature, target]
17            r, p = stats.spearmanr(corr_df[feature], corr_df[target]) #spearman correlation
18            n = n - 1
19        except Exception as e:
20            n = n - 1
21
22    return temp_df
23
24

```



특징별로 WOE 계산 및 실행

```

1 #execute woe for all the variables in the dataset
2 def execute_woe(df, target):
3
4     count = -1
5     for feature in final_vars:
6         #execute if the feature is not a target column name. Provided as an extra check.
7         if feature != target:
8             count = count + 1
9             temp_df = df.select([feature, target])
10
11            #perform monotonic binning for numeric variables before doing woe calculation
12            if feature in num_vars:
13                temp_df = mono_bin(temp_df, feature, target, n = max_bin)
14                # group buckets in numerical
15                grouped = temp_df.groupby('buckets')
16            else:
17                # just group categories in categorical
18                grouped = temp_df.groupby(feature)
19
20            #count and event value for each group
21            count_df = grouped.agg(F.count(target).alias('count')).toPandas()
22            event_df = grouped.agg(F.sum(target).alias('event')).toPandas()
23
24            #store min and max values for variables. for category both takes the same value.
25            if feature in num_vars:
26                min_value = grouped.agg(F.min(feature).alias('min')).toPandas()['min']
27                max_value = grouped.agg(F.max(feature).alias('max')).toPandas()['max']
28            else:
29                min_value = count_df[feature]
30                max_value = count_df[feature]
31
32            #calculate woe and iv
33            temp_woe_df = calculate_woe(count_df, event_df, min_value, max_value, feature)
34
35            #final dataset creation
36            if count == 0:
37                final_woe_df = temp_woe_df
38            else:
39                final_woe_df = final_woe_df.append(temp_woe_df, ignore_index=True)
40
41            # separate IV dataset creation
42            iv = pd.DataFrame({'IV':final_woe_df.groupby('varname').iv.max()})
43            iv = iv.reset_index()
44    return final_woe_df, iv

```

WOE & IV 수행

```
[24] 1 # invoke woe & iv code
      2 output, iv = execute_woe(df, target_variable_name)

[30] 1 pd.set_option('display.max_rows', output.shape[0]+1)
      2 print(output)

[26] 1 iv.sort_values('IV', ascending=False)
```

47	Rooms	1	...	3.201493	1.750996
48	Rooms	2	...	1.303277	1.750996
49	Rooms	4	...	-2.059230	1.750996
50	Rooms	3	...	-0.664474	1.750996
...	...	...	...	...	...
124	Landsize	361	...	-1.293926	2.835305
125	Landsize	312	...	-0.290767	2.835305
126	Landsize	930	...	0.782938	2.835305
127	Landsize	0	...	3.142397	2.835305
128	Landsize	168	...	0.397416	2.835305
129	Landsize	552	...	-3.239926	2.835305
130	Landsize	650	...	-2.744365	2.835305
131	Landsize	130	...	0.595711	2.835305
132	Landsize	37	...	1.145843	2.835305
133	Landsize	785	...	-1.212831	2.835305
134	Landsize	501	...	-2.415804	2.835305
135	Landsize	613	...	-2.656923	2.835305
136	Landsize	264	...	0.047628	2.835305
137	Landsize	214	...	0.303319	2.835305
138	Landsize	684	...	-2.662718	2.835305
139	Landsize	438	...	-1.941904	2.835305
140	Landsize	721	...	-1.890140	2.835305
141	Landsize	585	...	-2.772063	2.835305
...	...	...	...	...	...

	varname	IV
6	Landsize	2.835305
14	Rooms	1.750996
1	Bedroom2	1.633744
11	Price	1.551814
2	BuildingArea	0.884353
15	YearBuilt	0.851490
3	Car	0.843557
4	CouncilArea	0.569996
5	Distance	0.340613
10	Postcode	0.270258
8	Longitude	0.259286
7	Latitude	0.229285
13	Regionname	0.195509
0	Bathroom	0.166885
12	Propertycount	0.071739
9	Method	0.035096

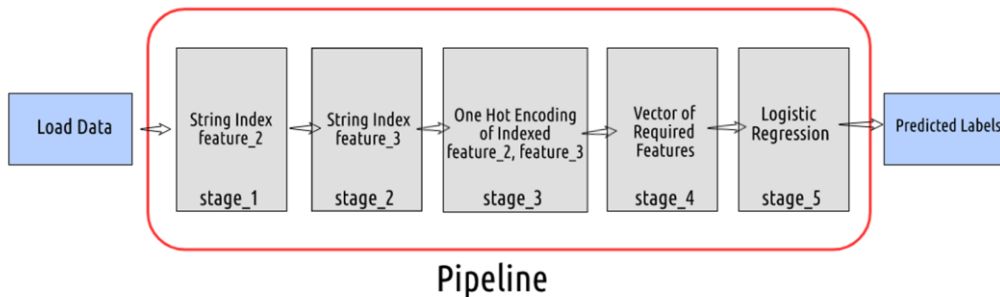


Custom Transformer

Custom Transformer

■ Pyspark Pipeline Model

- 다수의 Stage로 구성하여 결과 도출
- OOP 개념을 사용하여, 워크플로우/파이프라인을 구축
 - End-to-end process

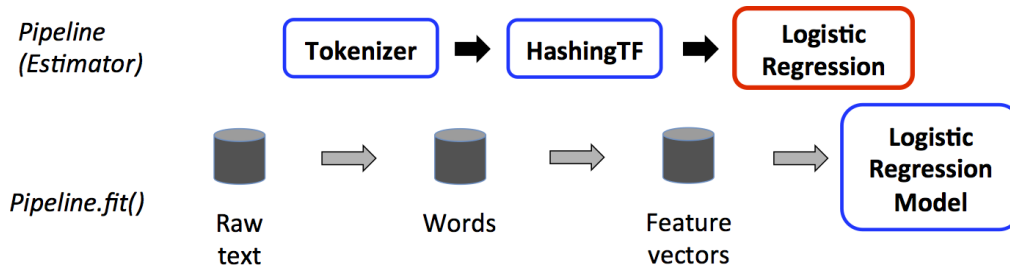


파이프라인의 주요 개념

- **DataFrame**: This ML API uses DataFrame from Spark SQL as an ML dataset, which can hold a variety of data types.
 - E.g., a DataFrame could have different columns storing text, feature vectors, true labels, and predictions
- **Transformer**: is an algorithm which can transform one DataFrame into another DataFrame.
 - E.g., an ML model is a Transformer which transforms a DataFrame with features into a DataFrame with predictions.
- **Estimator**: is an algorithm which can be fit on a DataFrame to produce a Transformer.
 - E.g., a learning algorithm is an Estimator which trains on a DataFrame and produces a model.
- **Pipeline**: chains multiple Transformers and Estimators together to specify an ML workflow.
- **Parameter**: All Transformers and Estimators now share a common API for specifying parameters.



파이프라인의 예제



```
# Configure an ML pipeline, which consists of three stages: tokenizer, hashingTF, and lr.
tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(inputCol=tokenizer.getOutputCol(), outputCol="features")
lr = LogisticRegression(maxIter=10, regParam=0.001)
pipeline = Pipeline(stages=[tokenizer, hashingTF, lr])
```



Custom Correlation

```
[ ] 1 from pyspark.mllib.stat import Statistics
    2
    3 correlation_type = 'pearson' # 'pearson', 'spearman'

[163] 1 features_df = None
    2 for k, v in df.schema["features"].metadata["ml_attr"]["attrs"].items():
    3     if features_df is None:
    4         features_df = pd.DataFrame(v)
    5     else:
    6         features_df = pd.concat([features_df, pd.DataFrame(v)], axis=0)
    7
    8 column_names = list(features_df['name'])

[164] 1 column_names

[165] 1 df_vector = df.rdd.map(lambda x: x['features'].toArray())

[166] 1 matrix = Statistics.corr(df_vector, method=correlation_type)
```



Custom Correlation

```
1 corr_df = pd.DataFrame(matrix, columns=column_names, index=column_names)
2 corr_df
```

	age	balance	day	duration	campaign	pdays	previous	job	marital	education	default	housing	loan	contact	month	poutcome
age	1.000000	0.097783	-0.009120	-0.004648	0.004760	-0.023758	0.001288	0.077468	-0.126351	0.167296	-0.017879	0.185513	-0.015655	0.122114	0.089717	0.012238
balance	0.097783	1.000000	0.004503	0.021560	-0.014578	0.003435	0.016674	0.020404	-0.028172	0.039067	-0.066745	0.068768	-0.084350	0.002844	0.092853	0.037272
day	-0.009120	0.004503	1.000000	-0.030206	0.162490	-0.093044	-0.051710	-0.010874	-0.005217	-0.004675	0.009424	0.027982	0.011370	-0.006302	-0.038019	-0.072629
duration	-0.004648	0.021560	-0.030206	1.000000	-0.084570	-0.001565	0.001203	0.004410	0.018834	-0.003029	-0.010021	-0.005075	-0.012412	-0.029350	0.014097	0.023192
campaign	0.004760	-0.014578	0.162490	-0.084570	1.000000	-0.088628	-0.032855	-0.025450	-0.029294	0.018643	0.016822	0.023599	0.009980	0.046971	-0.093829	-0.094982
pdays	-0.023758	0.003435	-0.093044	-0.001565	-0.088628	1.000000	0.454820	-0.020621	-0.020832	-0.023103	-0.029979	-0.124178	-0.022754	-0.170654	0.130504	0.709008
previous	0.001288	0.016674	-0.051710	0.001203	-0.032855	0.454820	1.000000	-0.002425	0.006876	-0.007463	-0.018329	-0.037076	-0.011043	-0.091911	0.124014	0.485040
job	0.077468	0.020404	-0.010874	0.004410	-0.025450	-0.020621	-0.002425	1.000000	0.050184	-0.002546	-0.003261	0.175678	-0.035331	0.011959	0.116516	0.022384
marital	-0.126351	-0.028172	-0.005217	0.018834	-0.029294	0.020832	0.006876	0.050184	1.000000	-0.061597	0.018404	0.011671	-0.018526	-0.038869	0.010914	0.020126
education	0.167296	0.039067	-0.004675	-0.003029	0.018643	-0.023103	-0.007463	-0.002546	-0.061597	1.000000	-0.007520	0.076023	-0.068765	0.062967	0.028758	-0.010689
default	-0.017879	-0.066745	0.009424	-0.010021	0.016822	-0.029979	-0.018329	-0.003261	0.018404	-0.007520	1.000000	0.006025	0.077234	0.000961	-0.033731	-0.037940
housing	0.185513	0.068768	0.027982	-0.005075	0.023599	-0.124178	-0.037076	0.175678	0.011671	0.076023	0.006025	1.000000	-0.041323	-0.089783	0.270254	0.000527
loan	-0.015655	-0.084350	0.011370	-0.012412	0.009980	-0.022754	-0.011043	-0.035331	-0.018526	-0.068765	0.077234	-0.041323	1.000000	-0.015964	-0.060078	-0.047586
contact	0.122114	0.002844	-0.006302	-0.029350	0.046971	-0.170654	-0.091911	0.011959	-0.038869	0.062967	0.000961	-0.089783	-0.015964	1.000000	-0.175432	-0.169951
month	0.089717	0.092853	-0.038019	0.014097	-0.093829	0.130504	0.124014	0.116516	0.010914	0.028758	-0.033731	0.270254	-0.060078	-0.175432	1.000000	0.234792
poutcome	0.012238	0.037272	-0.072629	0.023192	-0.094982	0.709008	0.485040	0.022384	0.020126	-0.010689	-0.037940	0.000527	-0.047586	-0.169951	0.234792	1.000000



Custom Correlation

```
1 final_corr_df = pd.DataFrame(corr_df.abs().unstack().sort_values(kind='quicksort')).reset_index()
2 final_corr_df.rename({'level_0': 'col1', 'level_1': 'col2', 0: 'correlation_value'}, axis=1, inplace=True)
3 final_corr_df = final_corr_df[final_corr_df['col1'] != final_corr_df['col2']]
4 final_corr_df
```

	col1	col2	correlation_value
0	housing	poutcome	0.000527
1	poutcome	housing	0.000527
2	default	contact	0.000961
3	contact	default	0.000961
4	duration	previous	0.001203
...	...	...	...
235	previous	pdays	0.454820
236	poutcome	previous	0.485040
237	previous	poutcome	0.485040
238	poutcome	pdays	0.709008
239	pdays	poutcome	0.709008

240 rows x 3 columns



Custom Correlation



Custom Correlation

```

1 from pyspark.ml import Transformer
2 from pyspark.ml.param.shared import HasInputCol
3 from pyspark.mllib.stat import Statistics
4
5 class CustomCorrelation(Transformer, HasInputCol):
6     """
7     A custom function to calculate the correlation between two variables.
8
9     Parameters:
10     -----
11     InputCol: default value (None)
12         Feature column name to be used for the correlation purpose. The input column should be assembled vector
13
14     correlation_type: 'pearson' or 'spearman'
15
16     correlation_cutoff: float, default value (0.7), accepted values 0 to 1
17         Columns more than the specified cutoff will be displayed in the output dataframe.
18     """
19
20     # Initialize parameters for the function
21     def __init__(self, inputCol=None, correlation_type='pearson', correlation_cutoff=0.7):
22
23         super(CustomCorrelation, self).__init__()
24
25         assert inputCol, "Please provide a assembled feature column name"
26
27         #self.inputCol is class parameter
28         self.inputCol = inputCol
29         assert correlation_type == 'pearson' or correlation_type == 'spearman', "Please provide #
30             a valid option for correlation type. 'pearson' or 'spearman'."
31
32         #self.correlation_type is class parameter
33         self.correlation_type = correlation_type
34         assert 0.0 <= correlation_cutoff <= 1.0, "Provide a valid value for cutoff. Accepted range is 0 to 1"
35

```

Custom Correlation

```

36     #self.correlation_cutoff is class parameter
37     self.correlation_cutoff = correlation_cutoff
38
39
40     # Transformer function, method inside a class, '_transform' - protected parameter
41     def _transform(self, df):
42         features_df = None
43         for k, v in df.schema["features"].metadata["ml_attr"]["attrs"].items():
44             if features_df is None:
45                 features_df = pd.DataFrame(v)
46             else:
47                 features_df = pd.concat([features_df, pd.DataFrame(v)], axis=0)
48
49         #self.column_names is class parameter, created for future use
50         self.column_names = list(features_df['name'])
51
52         df_vector = df.rdd.map(lambda x: x[self.inputCol].toArray())
53         matrix = Statistics.corr(df_vector, method=self.correlation_type)
54
55         # apply pandas dataframe operation on the fit output
56         corr_df = pd.DataFrame(matrix, columns=self.column_names, index=self.column_names)
57
58         final_corr_df = pd.DataFrame(corr_df.abs().unstack().sort_values(kind='quicksort')).reset_index()
59         final_corr_df.rename({'level_0': 'col1', 'level_1': 'col2', 0: 'correlation_value'}, axis=1, inplace=True)
60         final_corr_df = final_corr_df[final_corr_df['col1'] != final_corr_df['col2']]
61
62         #shortlisted dataframe based on custom cutoff
63         shortlisted_corr_df = final_corr_df[final_corr_df['correlation_value'] > self.correlation_cutoff]
64
65         return corr_df, shortlisted_corr_df

```



Custom Correlation

```

1 clf = CustomCorrelation(inputCol='features')
2 output, shortlisted_output = clf.transform(df)

```

output

	age	balance	day	duration	campaign	pdays	previous	job	marital	education	default	housing	loan	contact	month	poutcome
age	1.000000	0.097783	-0.009120	-0.004648	0.004760	-0.023758	0.001288	0.077468	-0.126351	0.167296	-0.017879	0.185513	-0.015655	0.122114	0.089717	0.012238
balance	0.097783	1.000000	0.004503	0.021560	-0.014578	0.003435	0.016674	0.020404	-0.028172	0.039067	-0.066745	0.068768	-0.084350	0.002844	0.092853	0.037272
day	-0.009120	0.004503	1.000000	-0.030206	0.162490	-0.093044	-0.051710	-0.010874	-0.005217	-0.004675	0.009424	0.027982	0.011370	-0.006302	-0.038019	-0.072629
duration	-0.004648	0.021560	-0.030206	1.000000	-0.084570	-0.001565	0.001203	0.004410	0.018834	-0.003029	-0.010021	-0.005075	-0.012412	-0.029350	0.014097	0.023192
campaign	0.004760	-0.014578	0.162490	-0.084570	1.000000	-0.088628	-0.032855	-0.025450	-0.029294	0.018643	0.016822	0.023599	0.009980	0.046971	-0.093829	-0.094982
pdays	-0.023758	0.003435	-0.093044	-0.001565	-0.088628	1.000000	0.454820	-0.020621	0.020832	-0.023103	-0.029979	-0.124178	-0.022754	-0.170654	0.130504	0.709008
previous	0.001288	0.016674	-0.051710	0.001203	-0.032855	0.454820	1.000000	-0.002425	0.006876	-0.007463	-0.018329	-0.037076	-0.011043	-0.091911	0.124014	0.485040
job	0.077468	0.020404	-0.010874	0.004410	-0.025450	-0.020621	-0.002425	1.000000	0.050184	-0.002546	-0.003261	0.175678	-0.035331	0.011959	0.116516	0.022384
marital	-0.126351	-0.028172	-0.005217	0.018834	-0.029294	0.020832	0.006876	0.050184	1.000000	-0.061597	0.018404	0.011671	-0.018526	-0.038869	0.010914	0.020126
education	0.167296	0.039067	-0.004675	-0.003029	0.018643	-0.023103	-0.007463	-0.002546	-0.061597	1.000000	-0.007520	0.076023	-0.068765	0.062967	0.028758	-0.010689
default	-0.017879	-0.066745	0.009424	-0.010021	0.016822	-0.029979	-0.018329	-0.003261	0.018404	-0.007520	1.000000	0.006025	0.077234	0.000961	-0.033731	-0.037940
housing	0.185513	0.068768	0.027982	-0.005075	0.023599	-0.124178	-0.037076	0.175678	0.011671	0.076023	0.006025	1.000000	-0.041323	-0.089783	0.270254	0.000527
loan	-0.015655	-0.084350	0.011370	-0.012412	0.009980	-0.022754	-0.011043	-0.035331	-0.018526	-0.068765	0.077234	-0.041323	1.000000	-0.015964	-0.060078	-0.047586
contact	0.122114	0.002844	-0.006302	-0.029350	0.046971	-0.170654	0.091911	0.011959	-0.038869	0.062967	0.000961	-0.089783	-0.015964	1.000000	-0.175432	-0.169951
month	0.089717	0.092853	-0.038019	0.014097	-0.093829	0.130504	0.124014	0.116516	0.010914	0.028758	-0.033731	0.270254	-0.060078	-0.175432	1.000000	0.234792
poutcome	0.012238	0.037272	-0.072629	0.023192	-0.094982	0.709008	0.485040	0.022384	0.020126	-0.010689	-0.037940	0.000527	-0.047586	-0.169951	0.234792	1.000000

[173] 1 output

shortlisted_output

	col1	col2	correlation_value
238	poutcome	pdays	0.709008
239	pdays	poutcome	0.709008



Pipeline for custom transformers

```

1 # features 컬럼을 삭제.
2 df = df.drop('features')

1 #exclude target variable and select all other feature vectors
2 features_list = df.columns
3 features_list.remove(target_variable_name)

1 # from pyspark.ml.feature import VectorAssembler
2 # from pyspark.ml import Pipeline
3
4 stages = []
5
6 # assemble vectors
7 assembler = VectorAssembler(inputCols=features_list, outputCol='features')
8 custom_corr = CustomCorrelation(inputCol=assembler.getOutputCol())
9 stages = [assembler, custom_corr]

1 #use pipeline to process sequentially
2 pipeline = Pipeline(stages=stages)

1 #pipeline model
2 pipelineModel = pipeline.fit(df)

1 #apply pipeline model on data
2 output, shortlisted_output = pipelineModel.transform(df)

```



Voting-based Selection


```

1 # decision tree
2 from pyspark.ml.classification import DecisionTreeClassifier
3 dt = DecisionTreeClassifier(featuresCol='features', labelCol=target_variable_name)
4 dt_model = dt.fit(df)
5 dt_output = dt_model.featureImportances
6 features_df['Decision Tree'] = features_df['idx'].apply(lambda x: dt_output[x] if x in dt_output.indices else 0)

1 # Gradient boosting
2 from pyspark.ml.classification import GBClassifier
3 gb = GBClassifier(featuresCol='features', labelCol=target_variable_name)
4 gb_model = gb.fit(df)
5 gb_output = gb_model.featureImportances
6 features_df['Gradient Boosting'] = features_df['idx'].apply(lambda x: gb_output[x] if x in gb_output.indices else 0)

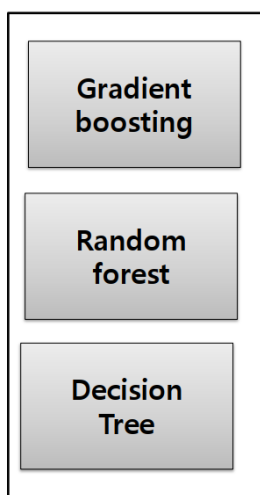
1 # Random forest
2 from pyspark.ml.classification import RandomForestClassifier
3 rf = RandomForestClassifier(featuresCol='features', labelCol=target_variable_name)
4 rf_model = rf.fit(df)
5 rf_output = rf_model.featureImportances
6 features_df['Random Forest'] = features_df['idx'].apply(lambda x: rf_output[x] if x in rf_output.indices else 0)

```



Overview

Features
Target_variable



Vote

Features
Target_variable
Score



Results

	name	Decision Tree	Gradient Boosting	Random Forest	final_score
3	duration	1	1	1	3
4	housing	1	1	1	3
6	contact	1	1	1	3
7	month	1	1	1	3
8	poutcome	1	1	1	3
2	day	1	1	0	2
5	pdays	0	1	1	2
0	age	0	0	1	1
1	balance	1	0	0	1
4	campaign	0	0	0	0
6	previous	0	0	0	0
0	job	0	0	0	0
1	marital	0	0	0	0
2	education	0	0	0	0
3	default	0	0	0	0
5	loan	0	0	0	0



Methods

```

1 # decision tree
2 from pyspark.ml.classification import DecisionTreeClassifier
3 dt = DecisionTreeClassifier(featuresCol='features', labelCol=target_variable_name)
4 dt_model = dt.fit(df)
5 dt_output = dt_model.featureImportances
6 features_df['Decision Tree'] = features_df['idx'].apply(lambda x: dt_output[x] if x in dt_output.indices else 0)

```

```

1 # Gradient boosting
2 from pyspark.ml.classification import GBClassifier
3 gb = GBClassifier(featuresCol='features', labelCol=target_variable_name)
4 gb_model = gb.fit(df)
5 gb_output = gb_model.featureImportances
6 features_df['Gradient Boosting'] = features_df['idx'].apply(lambda x: gb_output[x] if x in gb_output.indices else 0)

```

```

1 # Random forest
2 from pyspark.ml.classification import RandomForestClassifier
3 rf = RandomForestClassifier(featuresCol='features', labelCol=target_variable_name)
4 rf_model = rf.fit(df)
5 rf_output = rf_model.featureImportances
6 features_df['Random Forest'] = features_df['idx'].apply(lambda x: rf_output[x] if x in rf_output.indices else 0)

```



Voting-based selection

```

1 features_df.drop(columns=['idx'], axis=1, inplace=True)

1 # Voting-based selection
2 num_top_features = 7
3 columns = ['Decision Tree', 'Gradient Boosting', 'Random Forest']
4 score_table = pd.DataFrame({}, [])
5 score_table['name'] = features_df['name']
6 for i in columns:
7     score_table[i] = features_df['name'].isin(list(features_df.nlargest(num_top_features, i)['name'])).astype(int)
8
9 score_table['final_score'] = score_table.sum(axis=1)
10 score_table.sort_values('final_score', ascending=0)

```



Summary

- 처음 데이터셋을 받았을 때,
 - 초기 전처리과정 (결측값, 오류 확인 등)
 - 변수 선택을 신중하게 해야함
 - 타겟값이 주어질때, 주어지지 않을때로 분리하여 생각할 것
- 비슷한 데이터셋을 계속 받는다면,
 - 파이프라인을 설계해보자



Singular Value Decomposition

Appendix #1

127

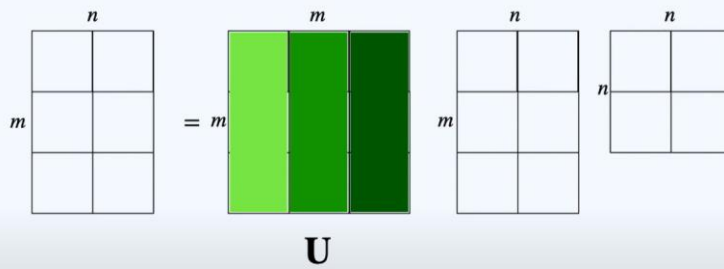
Singular value decomposition (SVD)



- Consider a 2-dimensional array $\mathbf{A} \in \mathbb{R}^{m \times n}, m > n$

$$\begin{array}{c}
 \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline & \\ \hline \end{array}
 \end{array}
 \begin{array}{c}
 n \\
 \\
 m
 \end{array}
 \mathbf{A}
 =
 \begin{array}{c}
 \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array}
 \end{array}
 \begin{array}{c}
 m \\
 \\
 m
 \end{array}
 \mathbf{U}
 \begin{array}{c}
 \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline & \\ \hline \end{array}
 \end{array}
 \begin{array}{c}
 n \\
 \\
 m
 \end{array}
 \mathbf{S}
 \begin{array}{c}
 \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline & \\ \hline \end{array}
 \end{array}
 \begin{array}{c}
 n \\
 \\
 n
 \end{array}
 \mathbf{V}^T$$

Singular value decomposition (SVD)

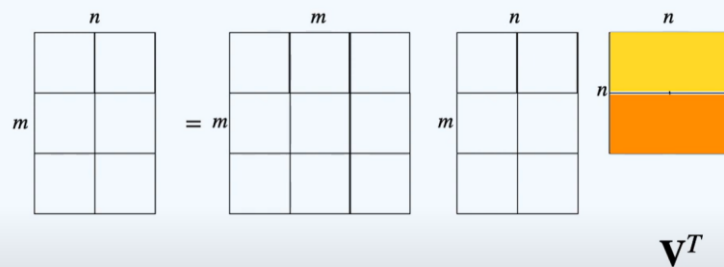


Left singular vectors

$$\mathbf{U}^{-1} = \mathbf{U}^T$$



Singular value decomposition (SVD)

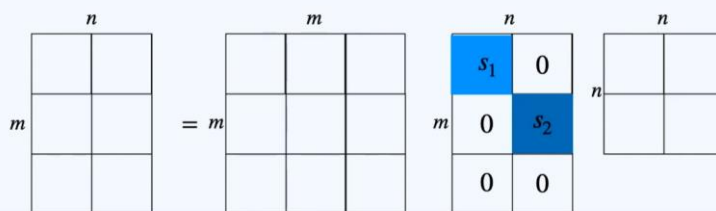


Right singular vectors

$$\mathbf{V}^{-1} = \mathbf{V}^T$$



Singular value decomposition (SVD)



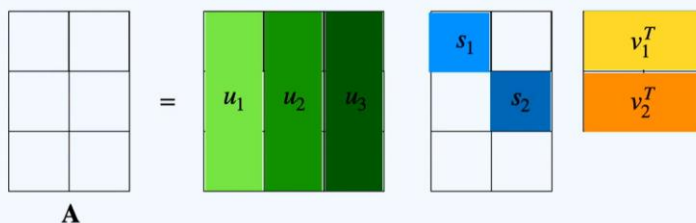
S

singular values

$$s_1 \geq s_2 \geq \dots$$



Application of SVD



$$= \begin{bmatrix} s_1 & & \\ & & \end{bmatrix} \begin{bmatrix} u_1 \\ & \end{bmatrix} + \begin{bmatrix} s_2 & & \\ & & \end{bmatrix} \begin{bmatrix} u_2 \\ & \end{bmatrix} + \dots$$



References

- Easiest Way to Understanding Singular Value Decomposition (SVD) with Python: `numpy.linalg.svd`
- <https://www.youtube.com/watch?v=5d67893GJao&t=1339s>



Information Value and Weight of Evidence

Appendix #2

Weight of Evidence(WoE)

- Feature의 예측력을 이해

$$WOE = \ln \left(\frac{\text{Distribution of Goods}}{\text{Distribution of Bads}} \right)$$

$$WoE = \log \left(\frac{p(Y = 1)}{p(Y = 0)} \right)$$



Information Value

- 해당 Feature의 예측력을 하나의 값으로 표현

$$IV = \sum (\text{Event\%} - \text{Non Event\%}) * (WOE)$$

Information Value (IV)	Predictive Power
< 0.02	useless for prediction
0.02 to 0.1	weak predictor
0.1 to 0.3	medium predictor
0.3 to 0.5	strong predictor
> 0.5	suspicious or too good to be true



Weight of Evidence(WoE)

■ WoE for a specific class

Age group	Good(1)	Bad(0)	%Good	%Bad	WOE	%G - %B	IV
20-30	2000	400					
30-40	3500	201					
40-50	4200	300					
60-70	1200	96					
Total							



Correlation/Coefficient

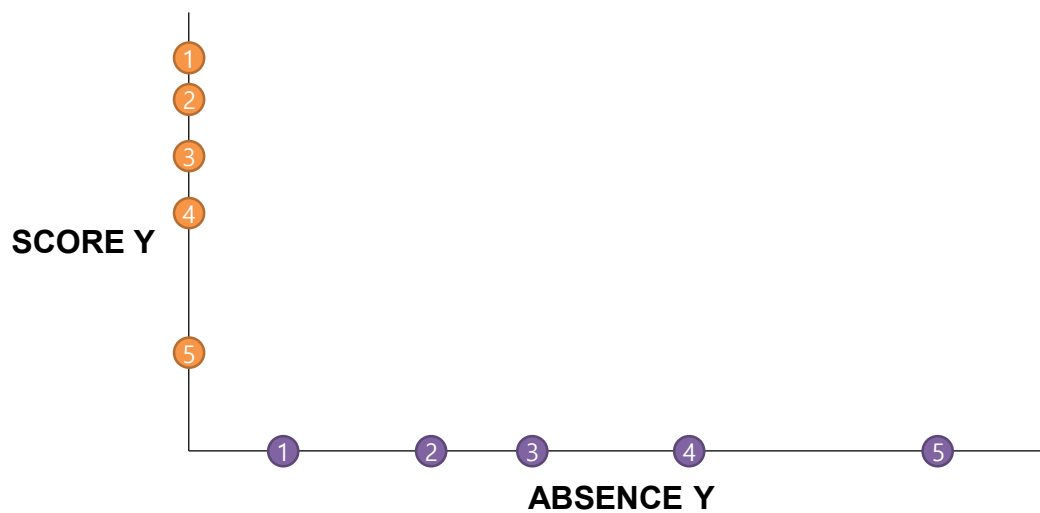
Appendix #3

Correlation

- measures the strength of association between two variables and the direction of the relationship
 - Pearson, Spearman, Kendal
- Also called “coefficient”



Non-parametric testing



Parametric testing

■ Correlation vs p-value

